Università degli Studi di Milano
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
GIOVANNI DEGLI ANTONI



CORSO DI LAUREA TRIENNALE IN
INFORMATICA

IMPLEMENTAZIONE DIGITALE DI UN GIOCO DA TAVOLO CON GRAFICA INTERATTIVA PER IL POTENZIAMENTO DELLE COMPETENZE SOCIALI DI ADOLESCENTI AUTISTICI

Relatore: Prof. Nunzio Alberto Borghese

Elaborato Finale di:
Samuel De Benedictis
Matr. Nr. 855264

Anno Accademico 2025-2026

Ringraziamenti

Ringrazio la dottoressa Daverio e tutta l'équipe di psicologi ed educatori del CFPIL di Varese per la fiducia, il tempo e il confronto costante che hanno accompagnato questo lavoro.

Un ringraziamento particolare va ai ragazzi del gruppo, ideatori de *La Città degli Imprevisti*: il loro entusiasmo e le loro idee sono alla base di questo progetto.

Indice

Ringraziamenti **i**

Indice **ii**

1 Introduzione **1**
1.1 L'autismo . . . 1
1.2 Il CFPIL di Varese . . . 2
1.3 Competenze sociali negli adolescenti autistici . . . 2
1.4 Il gioco da tavolo . . . 3
1.5 *La Città degli Imprevisti* . . . 4
1.5.1 Origine del progetto . . . 4
1.5.2 Dal prototipo cartaceo al videogioco . . . 5
1.5.3 Un gruppo in evoluzione . . . 5
1.5.4 Le caselle speciali . . . 5
1.5.5 Sviluppi previsti . . . 7

2 Stato dell'arte **8**
2.1 Approcci al social skills training nell'ASD . . . 8
2.2 Il gioco come strumento di intervento . . . 8
2.3 Serious games digitali: stato delle evidenze . . . 9
2.4 Posizionamento del presente lavoro . . . 9

3 Tecnologie utilizzate **11**
3.1 Framework e librerie di sviluppo . . . 11
3.1.1 Next.js . . . 11
3.1.2 TypeScript . . . 11
3.1.3 React . . . 12
3.1.4 Zustand . . . 12
3.1.5 Vercel . . . 12
3.1.6 Upstash Redis . . . 12
3.1.7 Tailwind CSS . . . 12

3.2 Strumenti di qualità del codice . . . 13
3.2.1 Biome . . . 13
3.2.2 Husky e Commitlint . . . 13
3.3 Strumenti di testing e documentazione . . . 13
3.3.1 Vitest . . . 13
3.3.2 Playwright . . . 13
3.3.3 Storybook . . . 13
3.4 Strumenti per la generazione dei contenuti . . . 14
3.4.1 MediaPipe . . . 14
3.4.2 OpenCV, SciPy e imageio . . . 14

4 Implementazione **15**
4.1 Contesto d'uso e scelte progettuali . . . 15
4.2 Architettura generale . . . 16
4.2.1 Evoluzione dall'architettura statica alla versione server-side . . . 16
4.3 Storyboard: l'uso dell'applicazione da parte dell'educatore . . . 17
4.3.1 Fase 1 — Preparazione della partita . . . 17
4.3.2 Fase 2 — Sessione a schermo singolo . . . 18
4.3.3 Fase 3 — Sessione multi-dispositivo . . . 19
4.3.4 Fase 4 — Chiusura . . . 19
4.4 Le schermate dell'applicazione . . . 24
4.4.1 Schermata di gioco . . . 24
4.4.2 Menu di navigazione . . . 27
4.4.3 Schermata Istruzioni . . . 28
4.4.4 Schermata Modalità avanzata . . . 28
4.4.5 Salvataggio e ripristino della partita . . . 28
4.4.6 Schermata Feedback . . . 30
4.4.7 Dashboard di analisi dei feedback . . . 30
4.4.8 Modalità multi-dispositivo . . . 31
4.5 Architettura dei moduli . . . 34
4.5.1 Domain Model . . . 34
4.5.2 State Management . . . 36
4.5.3 Presentation Layer . . . 37
4.6 Scelte implementative rilevanti . . . 38
4.6.1 Il tipo `GameActionResult` e il Command Pattern . . . 38
4.6.2 Gestione dello stato con Zustand e persistenza . . . 40
4.7 Design pattern utilizzati . . . 41
4.7.1 Command Pattern . . . 41
4.7.2 Facade Pattern . . . 41
4.7.3 Factory Pattern . . . 41

4.7.4 Observer Pattern . . . 42
4.7.5 Manager Pattern . . . 42
4.7.6 Model-View-Controller . . . 42
4.8 Modalità multi-dispositivo . . . 42
4.8.1 Architettura delle sessioni . . . 42
4.8.2 Connessione dei giocatori tramite QR code . . . 44
4.8.3 Stato lato client: il ruolo ridotto di Zustand . . . 44
4.8.4 Comunicazione in tempo reale tramite *long polling* . . . 44
4.8.5 Rilevamento dell'inattività . . . 45
4.8.6 Svolgimento del turno in modalità multi-dispositivo . . . 45
4.8.7 Considerazioni sulla dimensione sociale . . . 46

5 Test **47**
5.1 Raccolta di feedback . . . 47
5.2 Risultati preliminari . . . 48

6 Conclusioni **49**
6.1 Riepilogo del lavoro . . . 49
6.2 Contributi . . . 49
6.3 Limitazioni . . . 50
6.4 Sviluppi futuri . . . 51

Bibliografia **53**

Capitolo 1

Introduzione

1.1 L'autismo

Il termine *"autistico"* nasce come aggettivo descrittivo: lo psichiatra svizzero Eugen Bleuler lo coniò per indicare la chiusura in sé stessi di alcuni suoi pazienti (dal greco *autos*). Solo in seguito fu adottato da altri autori per descrivere difficoltà simili osservate in bambini e adolescenti.

Già nel 1926 la psichiatra Grunya Efimovna Sukhareva aveva documentato con grande dettaglio le caratteristiche autistiche di sei bambini — eppure la diagnosi rimase quella di psicopatia schizoide. Ci vollero altri vent'anni prima che l'autismo venisse riconosciuto come condizione a sé.

Nel 1943, in modo indipendente, due ricercatori arrivarono a conclusioni simili. Hans Asperger, nella sua tesi di dottorato, descrisse la condizione di alcuni ragazzi basandosi sull'osservazione di circa duecento pazienti, usando il termine *"psicopatia autistica"*. Nello stesso anno Leo Kanner riportò le osservazioni su undici bambini con *"marcate difficoltà nelle relazioni sociali e nella comunicazione"* e, in un lavoro successivo, propose il nome *"autismo infantile"*.

Negli anni Settanta Lorna Wing mise in discussione l'idea che l'autismo potesse essere ricondotto a criteri diagnostici univoci: fornì le prime prove sistematiche della sua natura composita e introdusse l'espressione *"disturbo dello spettro autistico"* (ASD, dall'inglese *Autism Spectrum Disorder*).

Il termine *"spettro"* non è neutro. Come sottolinea Surian [1], riprende la concezione dimensionale già implicita nel lavoro di Asperger: la differenza tra persone con o senza autismo va pensata come una differenza di grado, non di qualità.

Nella pratica, la diagnosi viene posta quando una costellazione di tratti si manifesta nello stesso individuo in forma molto accentuata: propensione per la ripetizione di alcune attività, spiccata attenzione ai dettagli, difficoltà nell'accettare cambiamenti. È proprio su

uno di questi versanti — quello delle competenze sociali — che il presente lavoro intende intervenire.

1.2 Il CFPIL di Varese

Il *Centro Formazione Professionale Integrazione Lavorativa* (CFPIL), istituito dalla Provincia di Varese, rappresenta un punto di riferimento consolidato da oltre trent'anni per l'utenza con disabilità intellettiva, fisica e sensoriale. Nel 2002, il centro è stato integrato all'interno dell'"Agenzia Formativa della Provincia di Varese", divenuta successivamente Azienda Speciale nel dicembre 2009.

I progetti del CFPIL costituiscono una risposta strutturata alle esigenze specifiche di adolescenti, giovani e adulti con disabilità, offrendo percorsi formativi mirati e successivi inserimenti lavorativi.

L'attività formativa è progettata e monitorata attraverso équipe multidisciplinari composte da educatori professionali, psicologi e assistenti sociali. Questo approccio consente di predisporre curricoli formativi personalizzati, alcuni dei quali si realizzano in contesti lavorativi all'interno di realtà produttive, aziende private ed enti pubblici.

Il CFPIL svolge una funzione di mediazione tra le persone con disabilità e il mondo del lavoro, erogando un sistema integrato di servizi che comprende:

- Informazione, valutazione e orientamento professionale;
- Formazione professionale specializzata;
- Integrazione lavorativa;
- Monitoraggio post-assunzione.

Il centro promuove relazioni sistemiche con il tessuto sociale, scolastico, produttivo e istituzionale del territorio, al fine di definire e realizzare progetti formativi condivisi. Attraverso questa rete collaborativa, il CFPIL intende contribuire alla diffusione del principio della diversità come risorsa individuale e allo sviluppo di una "cultura della diversità" fondata sul rispetto e sull'inclusione. [2]

1.3 Competenze sociali negli adolescenti autistici

Il potenziamento delle competenze sociali degli utenti del CFPIL costituisce uno degli obiettivi primari dell'istituto, riconosciuto come condizione imprescindibile per favorire l'inserimento lavorativo e la partecipazione attiva alla vita comunitaria.

Il presente lavoro si rivolge in particolare a un gruppo di adolescenti di età compresa tra i 15 e i 19 anni, con un funzionamento cognitivo caratterizzato da un deficit di entità

lieve, associato alle difficoltà socio-relazionali tipiche dello spettro autistico. Questa combinazione si traduce spesso in incertezza nell'affrontare i problemi della vita quotidiana e in una complessa gestione dell'adattamento sociale.

Per molti adolescenti con disturbo dello spettro autistico le competenze sociali rappresentano un'area di significativa difficoltà. Con questo termine si fa riferimento alla capacità di interagire in modo adeguato al contesto, di comunicare bisogni e intenzioni, di rispettare turni e ruoli e di gestire situazioni di conflitto: abilità che risultano spesso più determinanti delle competenze tecniche ai fini dell'integrazione lavorativa.

Sul piano neuropsicologico, queste difficoltà derivano da una combinazione di fattori tra loro correlati: bassa motivazione spontanea a iniziare interazioni sociali; tendenza a interazioni routinizzate anziché reciproche; difficoltà nella lettura di regole sociali implicite, come il linguaggio del corpo, il tono della voce o i segnali di disinteresse dell'interlocutore; deficit di teoria della mente, ovvero la capacità di inferire pensieri e intenzioni altrui [3]. Quest'ultimo deficit può portare, ad esempio, a prolungare una conversazione su un argomento di interesse personale senza percepire il disinteresse del proprio interlocutore, o a non cogliere il significato di un commento sarcastico.

Il CFPIL persegue questo obiettivo principalmente attraverso laboratori pratici a carattere professionalizzante, nei quali l'attività manuale diventa il mezzo attraverso cui esercitare e consolidare abilità relazionali in contesti strutturati e realistici, con ruoli definiti e procedure sequenziali che riproducono le dinamiche del mondo del lavoro.

Lavorare in contesti reali e motivanti, con una logica procedurale, permette di sviluppare le competenze sociali in modo più efficace rispetto agli esercizi astratti — e favorisce la generalizzazione di quanto appreso al di fuori del contesto scolastico o terapeutico. A questo si aggiunge un dato non scontato: gli adolescenti autistici esprimono interesse per la socializzazione, e il gioco è uno dei contesti in cui questo interesse emerge con più facilità [4]. Il formato ludico diventa quindi un canale di intervento, non un ripiego.

1.4 Il gioco da tavolo

Per i bambini e gli adolescenti con disturbo dello spettro autistico, il gioco riveste un'importanza terapeutica significativa: rappresenta una cornice educativa motivante in cui sperimentare abilità, consolidare comportamenti e relazionarsi con le persone circostanti. Qualsiasi gioco, indipendentemente dal suo grado di serietà, insegna implicitamente ai partecipanti come comportarsi in un contesto di gruppo, risultando particolarmente motivante rispetto ad altre forme di intervento educativo [5]. Tra le varie forme di gioco, quella che raggiunge il più alto livello di strutturazione è il gioco da tavolo, caratterizzato dall'utilizzo di elementi fisici definiti (pedine, carte, tabelloni, dadi) e da un insieme di regole esplicite che delimitano uno spazio e un tempo di gioco ben determinati. Questa struttura chiusa e ben delimitata lo rende particolarmente adatto al contesto autistico. Le analogie con le attività di lavoro indipendente previste dalla metodologia *Treatment and Education*

of Autistic and related Communication handicapped Children (TEACCH) non sono casuali: entrambe si svolgono seduti al tavolo, sono strutturate nel tempo e nello spazio, hanno una fine prestabilita e privilegiano il codice visivo rispetto a quello verbale [3, 6]. Non a caso, tra le attività raccomandate dall'approccio TEACCH per il lavoro sulle competenze sociali di adolescenti figurano esplicitamente giochi da tavolo in gruppo e videogiochi [3].

A differenza delle attività di lavoro individuale, tuttavia, il gioco da tavolo richiede la partecipazione di almeno due soggetti, offrendo opportunità di interazione sociale più ricche rispetto alle alternative esclusivamente digitali [5].

Come approfondito nel capitolo 2, la letteratura sul gioco come strumento di *social skills training* per persone autistiche riporta esiti costantemente positivi [4], ma si è concentrata su strumenti digitali sviluppati appositamente: le implementazioni digitali di giochi da tavolo strutturati restano un approccio ancora poco esplorato.

L'implementazione digitale descritta in questo elaborato si colloca in questo spazio. Rispetto alla versione fisica, il formato digitale consente di adattare parametri come il numero di caselle, la tipologia delle sfide o il numero di giocatori direttamente dall'interfaccia — senza dover riprogettare e ristampare i componenti. L'interazione, però, rimane quella del gioco da tavolo: faccia a faccia, attorno a uno schermo condiviso.

1.5 *La Città degli Imprevisti*

1.5.1 Origine del progetto

La Città degli Imprevisti nasce all'interno dei percorsi del CFPIL come risposta strutturata ai bisogni evolutivi e relazionali del gruppo descritto nella sezione precedente. Il lavoro è partito da un modulo dedicato alla consapevolezza personale e di gruppo, che ha permesso ai partecipanti di esaminare le proprie caratteristiche individuali e di confrontarsi in un ambiente protetto. L'esito più significativo di questa fase è stato il ridimensionamento del concetto di "malattia" a favore della presa di coscienza del proprio specifico tipo di funzionamento, con benefici sull'autoaccettazione e sul confronto tra pari.

Dalla maturazione del gruppo è emersa l'esigenza di lavorare sull'adattamento quotidiano, in particolare sul *problem solving* e sulla decodifica delle emozioni altrui. L'intervento si è articolato nell'approfondimento della comunicazione verbale e di quella analogica (mimica, prossemica e gestualità) e in esercitazioni pratiche per individuare soluzioni rapide ed efficaci di fronte a ostacoli e imprevisti personali e sociali.

Per sintetizzare ed esercitare l'insieme di queste competenze, il gruppo ha ideato, insieme all'équipe di psicologi ed educatori, una modalità ludica strutturata. Con strumenti semplici — cartellone, pennarelli e ritagli di carta — è nato il prototipo da tavolo, una piattaforma di simulazione in cui applicare le strategie comunicative e risolutive apprese. La struttura di base è quella del Gioco dell'Oca — un percorso lineare, un dado, le pedine —

arricchita con caselle speciali che attivano minigiochi a carattere sociale ed emotivo. L'obiettivo è arrivare per primi all'ultima casella, ma lungo il percorso i giocatori si trovano a dover comunicare, collaborare, riconoscere emozioni — non come esercizio isolato, ma come parte del gioco stesso.

1.5.2 Dal prototipo cartaceo al videogioco

Dalla volontà del gruppo di condividere lo strumento all'esterno, e dall'iniziativa di uno dei partecipanti, che aveva appreso in autonomia le basi della programmazione, è nata l'idea di trasformare il tabellone cartaceo in un videogioco. La scelta del canale digitale risponde a due considerazioni segnalate dall'équipe: l'interesse che molte persone nello spettro dimostrano per l'informatica e l'età dei partecipanti, per i quali il gaming è un linguaggio naturale e un primario strumento di mediazione relazionale.

Il presente elaborato descrive la traduzione di quel prototipo in un'applicazione web interattiva, fruibile online e tramite smartphone, sviluppata a partire dall'ascolto dei bisogni dei partecipanti e dalle loro idee.

La digitalizzazione ha avuto un impatto positivo di valorizzazione personale: i partecipanti hanno visto il proprio lavoro riconosciuto e trasformato in un prodotto tecnologico reale e fruibile. Il gioco è stato presentato e condiviso con gli altri studenti del CFPIL, diventando uno strumento relazionale esteso e rispondendo al linguaggio espressivo tipico della loro fascia d'età.

1.5.3 Un gruppo in evoluzione

Nel corso del tempo la composizione del gruppo è cambiata: alcuni dei membri originali hanno concluso il percorso formativo per inserirsi nel mondo del lavoro, lasciando il posto a nuovi ingressi. I partecipanti "veterani" hanno assunto spontaneamente il ruolo di *mentor*, affiancando i nuovi compagni nel percorso sulla consapevolezza dell'autismo e introducendoli alle dinamiche del gioco. Secondo l'équipe, questo passaggio di testimone ha rafforzato l'efficacia del modello, rendendo il gruppo stesso un ambiente formativo in grado di sostenersi autonomamente. Si tratta di un'osservazione qualitativa, non misurata nel presente lavoro.

1.5.4 Le caselle speciali

Le caselle speciali costituiscono il cuore educativo del gioco e sono progettate per sollecitare competenze diverse:

- **Mimo** — Il giocatore deve mimare una parola o una frase; allena la comunicazione non verbale.

- **Quiz** — Viene posta una domanda a risposta aperta o a scelta multipla; stimola l'attenzione e il ragionamento.
- **Parole alle spalle** — Il giocatore traccia con il dito una parola sulla schiena di un compagno, che deve riconoscerla; promuove il contatto fisico guidato e la comunicazione tattile.
- **Disegno dettato** — Un giocatore descrive a parole un'immagine che un compagno deve riprodurre graficamente; esercita la comunicazione verbale precisa e la collaborazione.
- **Emozioni in musica** — Viene proposta un'emozione e il giocatore deve scegliere una canzone che la rappresenti; allena l'espressione emotiva.
- **Indovina l'emozione** — Viene mostrato un volto e il giocatore deve identificare l'emozione espressa; allena il riconoscimento delle emozioni attraverso il volto.
- **Film** — Viene riprodotta una breve scena di un film e il giocatore deve identificare l'emozione che trasmette; allena il riconoscimento delle emozioni a partire da una situazione in movimento, con voce e musica, anziché da un'immagine statica. Nella versione attuale la sfida è disattivata, per l'assenza dei diritti di utilizzo sulle scene (sezione 4.5.1).
- **Cosa faresti se…** — Il giocatore risponde a uno scenario ipotetico e gli altri valutano la risposta; allena la prospettiva altrui, una delle abilità più difficili da sviluppare nell'ASD.
- **Test fisico** — Viene proposta una prova motoria da eseguire; favorisce la coordinazione e la gestione del corpo nello spazio condiviso.
- **Battaglia** — A differenza delle altre tipologie, non è legata a una casella specifica: quando due pedine si trovano sulla stessa casella, indipendentemente dal suo tipo, i due giocatori devono sfidarsi a morra cinese (carta-forbice-sasso); introduce la competizione regolamentata e la gestione dell'esito (vittoria e sconfitta).
- **Caselle movimento** — Alcune caselle fanno avanzare o retrocedere il giocatore; replicano la meccanica classica del Gioco dell'Oca, introducendo l'elemento della fortuna e dell'attesa.

Alcune di queste tipologie trovano riscontro nella letteratura empirica: come riportato da Atherton e Cross [5], giochi di riconoscimento delle espressioni facciali hanno prodotto miglioramenti significativi in bambini con disturbo dello spettro autistico (Tanaka et al., 2010, citato in [5]); attività di mimica con contenuto emotivo hanno dimostrato di allenare

la decodifica del linguaggio non verbale (Dell'Angela et al., 2020, citato in [5]); il gioco di ruolo da tavolo con adolescenti autistici ha prodotto miglioramenti nel benessere emotivo e nelle amicizie (Katō, 2019, citato in [5]).

Le caselle speciali non sono scelte in modo arbitrario: nel loro insieme, coprono le aree di difficoltà strutturale che la letteratura individua come caratteristiche dell'ASD — lettura delle espressioni facciali e del linguaggio non verbale, decodifica dei gesti, assunzione della prospettiva altrui, comunicazione verbale precisa, gestione delle emozioni, collaborazione e competizione regolamentata [3]. La varietà delle sfide garantisce che durante la partita vengano esercitate competenze eterogenee, mantenendo alto il coinvolgimento dei partecipanti.

1.5.5 Sviluppi previsti

La collaborazione tra équipe e gruppo prosegue con una nuova fase, non ancora implementata, dedicata alla prevenzione dei pericoli sociali, come bullismo e tentativi di raggiro; è descritta tra gli sviluppi futuri (sezione 6.4).

Capitolo 2

Stato dell'arte

2.1 Approcci al social skills training nell'ASD

Il *social skills training* (SST) per persone con disturbo dello spettro autistico comprende un insieme eterogeneo di approcci: dall'istruzione individuale basata su tecniche di modifica del comportamento, ai programmi di gruppo strutturati, fino agli interventi mediati da pari e alle metodologie basate sulla routine visiva, come TEACCH [3]. Nonostante la varietà degli approcci e il numero crescente di studi empirici, non si è ancora affermata una pratica ampiamente condivisa [7]: i risultati tendono a essere positivi nel contesto di intervento, ma faticano a generalizzarsi ad altri ambienti. Una variabile critica trasversale a tutti gli approcci è la motivazione del partecipante: perché un intervento funzioni, il soggetto deve trovare nelle attività proposte uno stimolo sufficiente a volersi impegnare [3].

2.2 Il gioco come strumento di intervento

Negli ultimi anni il gioco ha acquisito centralità come canale di intervento per le competenze sociali nelle persone con ASD. Atherton e Cross [5] hanno documentato una vasta gamma di applicazioni — dai giochi analogici a quelli digitali — evidenziando come il formato ludico offra un contesto motivante in cui comportamenti sociali complessi possono essere esercitati in modo naturale, grazie ai meccanismi di avanzamento e ricompensa integrati nella struttura del gioco. Walsh, Linehan e Ryan [4] hanno analizzato specificamente l'uso del gioco come strumento di SST per bambini e adolescenti autistici: tutti gli studi inclusi nella revisione riportano esiti positivi, sebbene gli autori segnalino la necessità di un approccio teoricamente più fondato nella definizione degli obiettivi comportamentali. Un dato rilevante emerso dalla revisione è che tutti gli studi includevano una

componente tecnologica, a conferma di come la ricerca si sia orientata prevalentemente verso strumenti digitali.

2.3 Serious games digitali: stato delle evidenze

Nell'ambito degli strumenti digitali, i *serious games* — giochi il cui scopo primario non è l'intrattenimento ma la trasmissione di conoscenze o competenze [7] — hanno ricevuto attenzione crescente come intervento per l'ASD. Una revisione sistematica di Carneiro et al. [7], condotta su 149 studi iniziali e selezionandone 9 secondo criteri PICO e linee guida PRISMA, ha analizzato l'efficacia di serious games progettati per il potenziamento delle competenze sociali in bambini e adolescenti con ASD.

I giochi inclusi nella revisione erano tutti strumenti software sviluppati appositamente, centrati su domini come il riconoscimento delle emozioni, la decodifica delle espressioni facciali, il contatto visivo, la joint attention e la regolazione emotiva. Il 100% degli studi analizzati riporta miglioramenti significativi in almeno uno dei costrutti misurati; l'85% mostra miglioramenti nelle competenze sociali generali, e tutti gli studi riportano effetti positivi sul riconoscimento delle emozioni, sulla prospettiva altrui e sulle abilità comportamentali [7].

La revisione segnala tuttavia alcune limitazioni: campioni ridotti, eterogeneità nei protocolli, rischio di bias elevato nella maggior parte degli studi, e una copertura quasi esclusiva della fascia 7–10 anni. Nessuno degli studi analizzati riguardava implementazioni digitali di giochi da tavolo strutturati.

2.4 Posizionamento del presente lavoro

La letteratura si è concentrata quasi esclusivamente su strumenti di questo tipo, lasciando poco esplorato il gioco da tavolo in versione digitale. A differenza dei serious games recensiti da Carneiro et al. [7], progettati per uso individuale o mediato dallo schermo, *La Città degli Imprevisti* — nella sua modalità di base con schermo singolo condiviso — mantiene l'interazione faccia a faccia attorno a un tavolo condiviso e richiede la mediazione di un educatore. La modalità multi-dispositivo, introdotta nella versione corrente, modifica parzialmente questa dinamica consentendo a ogni giocatore di usare il proprio smartphone; le implicazioni di questa scelta sulla qualità dell'interazione sociale sono discusse nella sezione 4.8. I domini esercitati — riconoscimento delle emozioni, comunicazione non verbale, prospettiva altrui, collaborazione, gestione della competizione — coincidono con le aree di intervento prioritarie nell'ASD identificate dalla letteratura.

Due ulteriori elementi distinguono il lavoro. Il primo riguarda i destinatari: la revisione di Carneiro et al. [7] copre quasi esclusivamente la fascia 7–10 anni, mentre il gruppo

coinvolto è composto da adolescenti di 15–19 anni con funzionamento cognitivo lievemente compromesso. Il secondo riguarda il processo di progettazione: il gioco non è stato ideato da un gruppo di ricerca, ma dai partecipanti stessi insieme all'équipe di psicologi ed educatori, e la sua versione digitale nasce su loro richiesta.

Capitolo 3

Tecnologie utilizzate

L'applicazione ha attraversato due fasi di sviluppo: una versione **v1** puramente client-side e una versione **v2** con una componente server-side per la modalità multi-dispositivo (si veda la sezione 4.2.1). Le tecnologie client-side — Next.js, TypeScript, React, Zustand, Tailwind — sono rimaste invariate; alla v2 si aggiungono le API Routes, un database Redis (Upstash) e l'hosting su Vercel.

Gli strumenti adottati in entrambe le versioni sono stati scelti per garantire qualità del codice, efficienza di sviluppo e manutenibilità nel lungo periodo.

3.1 Framework e librerie di sviluppo

3.1.1 Next.js

Next.js è un framework React per applicazioni web che integra nativamente routing basato su file, rendering lato server (SSR), generazione di siti statici (SSG) e API Routes per la definizione di endpoint HTTP nel medesimo progetto. È stato scelto come framework principale per la sua flessibilità: la stessa base di codice ha supportato sia l'esportazione statica della v1 sia le API Routes della v2 (sezione 4.2.1).

3.1.2 TypeScript

TypeScript è un superset di JavaScript che aggiunge tipizzazione statica al linguaggio, consentendo di individuare errori a tempo di compilazione anziché a runtime. Nel contesto di un progetto con un domain model articolato — composto da classi, manager e interfacce — la tipizzazione ha reso più sicuro il refactoring e ha ridotto significativamente la categoria di bug legati a tipi inattesi.

3.1.3 React

React è la libreria su cui si basa Next.js per la costruzione dell'interfaccia utente tramite componenti dichiarativi. Il modello a componenti consente di strutturare l'interfaccia in unità indipendenti e riutilizzabili, ciascuna responsabile del proprio stato visivo e del proprio ciclo di vita.

3.1.4 Zustand

Zustand è una libreria leggera per la gestione dello stato globale dell'applicazione. A differenza di soluzioni più elaborate come Redux, non richiede *boilerplate* e si integra in modo naturale con TypeScript. Il middleware `persist` consente di sincronizzare automaticamente gli store con il `localStorage` del browser, rendendo lo stato della partita persistente al ricaricamento della pagina senza codice aggiuntivo.

3.1.5 Vercel

Vercel è la piattaforma di hosting e deployment su cui è distribuita la versione corrente dell'applicazione. Integra nativamente Next.js, gestendo sia la distribuzione dei componenti client-side sia l'esecuzione delle API Routes come funzioni serverless. Il deploy avviene automaticamente a ogni push sul ramo principale del repository GitHub.

3.1.6 Upstash Redis

Upstash Redis è il servizio di *key-value store* utilizzato in produzione per la persistenza delle sessioni multi-dispositivo. Si distingue da un'istanza Redis tradizionale per l'accesso tramite API HTTP anziché connessioni TCP persistenti, caratteristica che lo rende compatibile con l'ambiente serverless di Vercel, in cui le funzioni non mantengono stato tra un'invocazione e la successiva. Il client `@upstash/redis` legge le credenziali dalle variabili di ambiente `KV_REST_API_URL` e `KV_REST_API_TOKEN`.

3.1.7 Tailwind CSS

Tailwind CSS è un framework CSS *utility-first*: invece di definire classi semantiche personalizzate, si compone l'aspetto visivo applicando direttamente nel markup classi predefinite di basso livello. Questo approccio ha velocizzato lo sviluppo dell'interfaccia, garantendo coerenza visiva e semplificando la gestione dei layout responsivi.

3.2 Strumenti di qualità del codice

3.2.1 Biome

Biome è uno strumento che unifica in un unico pacchetto le funzionalità di linting, formattazione e organizzazione degli import, sostituendo la tradizionale combinazione ESLint + Prettier con una configurazione singola e tempi di esecuzione notevolmente inferiori. È configurato per essere eseguito automaticamente a ogni commit tramite Husky.

3.2.2 Husky e Commitlint

Husky gestisce i Git hooks, permettendo di eseguire automaticamente controlli prima che un commit venga registrato. Nel progetto è configurato per eseguire Biome (linting e formattazione) e Commitlint (validazione del messaggio di commit) a ogni `git commit`. Commitlint verifica che i messaggi rispettino la convenzione *Conventional Commits*, migliorando la leggibilità della cronologia del progetto.

3.3 Strumenti di testing e documentazione

3.3.1 Vitest

Vitest è un framework di test unitari progettato per integrarsi con l'ecosistema Vite/Next.js. Offre prestazioni superiori rispetto a Jest grazie all'esecuzione nativa dei moduli ES e supporta la misurazione della copertura del codice tramite Istanbul. Viene utilizzato per testare la logica del domain model in isolamento dall'interfaccia.

3.3.2 Playwright

Playwright è un framework per test end-to-end che automatizza browser reali (Chromium, Firefox, WebKit). Permette di verificare il comportamento completo dell'applicazione simulando le interazioni di un utente reale, coprendo scenari che i test unitari non possono rilevare.

3.3.3 Storybook

Storybook è uno strumento per lo sviluppo e la documentazione dei componenti UI in isolamento. Ogni componente è associato a una o più *stories* che ne mostrano i diversi stati e configurazioni, facilitando lo sviluppo visivo indipendente e costituendo una documentazione interattiva dell'interfaccia. Nella pipeline di build viene compilato come sito statico e pubblicato insieme all'applicazione, risultando accessibile pubblicamente; un collegamento diretto è presente nella schermata delle istruzioni.

3.4 Strumenti per la generazione dei contenuti

Le animazioni di transizione del mazzo "Indovina l'emozione" non sono realizzate a mano ma prodotte da uno script Python, separato dall'applicazione web e non eseguito a *runtime*: le gif vengono generate una volta e distribuite come file statici insieme all'applicazione.

3.4.1 MediaPipe

MediaPipe è una libreria di Google per la percezione basata su modelli di apprendimento automatico. Il modello *Face Landmarker* restituisce, per un volto in un'immagine, 478 punti di riferimento (contorno del viso, sopracciglia, occhi, naso, labbra, iridi). Il modello è distribuito come singolo file (`face_landmarker.task`, circa 3,7 MB) ed è eseguito in locale: le fotografie non vengono inviate a servizi esterni.

3.4.2 OpenCV, SciPy e imageio

OpenCV fornisce le operazioni sulle immagini (ridimensionamento, deformazione affine dei singoli triangoli, maschere sfumate); SciPy calcola la triangolazione di Delaunay dei punti di riferimento; imageio assembla i fotogrammi nella gif finale. L'ambiente è isolato in un ambiente virtuale Python, creato dal comando `make face-morph-setup`.

Capitolo 4

Implementazione

Le tecnologie descritte nel capitolo precedente si riflettono direttamente nelle scelte architetturali dell'applicazione. Questo capitolo descrive come sono state combinate per realizzare il sistema, a partire dal contesto d'uso fino ai dettagli implementativi.

4.1 Contesto d'uso e scelte progettuali

L'applicazione è progettata per sessioni di gruppo in presenza, guidate da un educatore. Tutti i partecipanti si trovano nella stessa stanza e condividono un unico dispositivo — tipicamente un tablet o un PC con schermo sufficientemente grande da essere visibile a tutti i giocatori seduti attorno al tavolo. Non vengono utilizzati dadi fisici: il lancio avviene premendo un pulsante sullo schermo, e il risultato viene animato e visualizzato dall'applicazione. Anche l'avanzamento delle pedine è gestito interamente dal software: dopo ogni lancio, la pedina del giocatore attivo si sposta automaticamente lungo il tabellone, casella dopo casella, con un breve balzo animato per ciascun passaggio, fino a raggiungere la casella di destinazione.

L'applicazione supporta due modalità operative. Nella modalità **schermo singolo**, tutti i partecipanti utilizzano un unico dispositivo condiviso: questa scelta preserva la natura faccia a faccia del gioco da tavolo originale, mantiene l'attenzione del gruppo centrata su un unico punto di riferimento e riduce la complessità infrastrutturale. Nella modalità **multi-dispositivo**, ogni giocatore utilizza il proprio smartphone come controller personale, mentre il tabellone rimane visibile su un dispositivo separato gestito dall'educatore (descritta nella sezione 4.8). Il ruolo dell'educatore rimane centrale in entrambe le modalità: è l'educatore a facilitare le sfide delle caselle speciali, a valutare le risposte dei giocatori e a registrare l'esito nell'applicazione, che ne applica gli effetti sul tabellone. Con "educatore" si intende chi conduce la sessione, figura che può essere un educatore o uno psicologo; il suo percorso d'uso è descritto nella sezione 4.3.

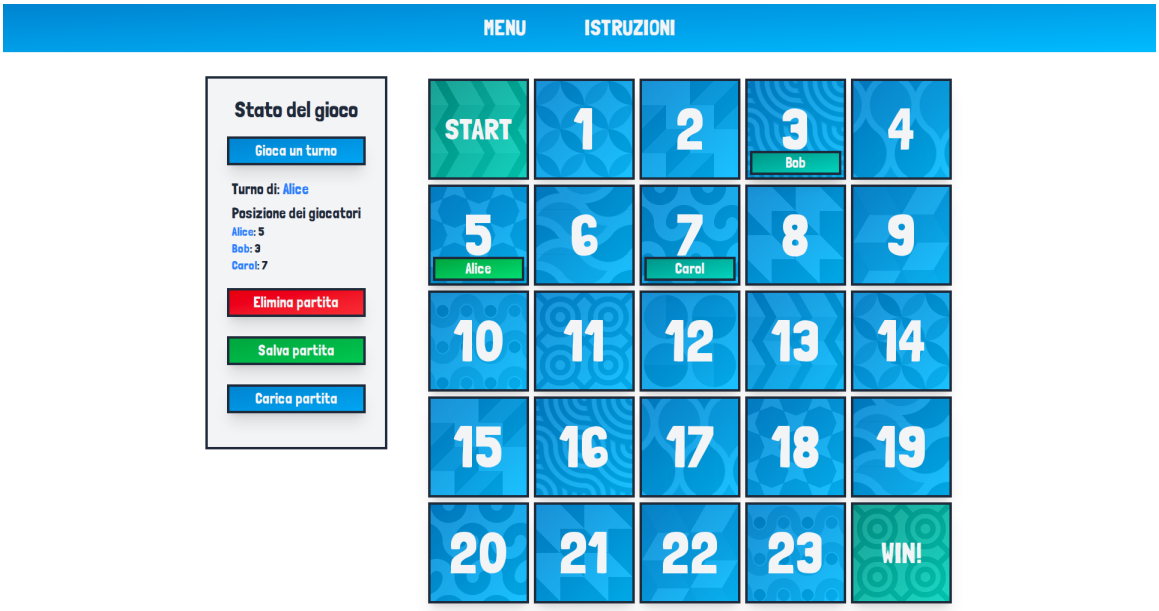


Figura 1: Schermata di gioco: tabellone, barra laterale e dado.

4.2 Architettura generale

L'applicazione è distribuita su Vercel come progetto Next.js a pieno titolo: comprende una parte client-side (componenti React, CSS, JavaScript) e una parte server-side costituita dalle API Routes che supportano la modalità multi-dispositivo. Non è richiesta alcuna installazione: l'applicazione è accessibile da qualsiasi browser moderno su PC, laptop, tablet o smartphone.

Nella modalità schermo singolo non vi è comunicazione di rete durante la partita: tutta la logica di gioco — compresa la gestione dello stato, il salvataggio della partita e la generazione del tabellone — risiede ed è eseguita nel browser dell'utente, con persistenza tramite `localStorage`. La modalità multi-dispositivo estende questa architettura con endpoint HTTP e uno store condiviso lato server, descritti nella sezione 4.8.

4.2.1 Evoluzione dall'architettura statica alla versione server-side

La prima versione dell'applicazione (**v1**, ultima release: 1.4.6) adottava la funzionalità di esportazione statica di Next.js: il processo di build produceva un insieme di file HTML, JavaScript e CSS distribuiti su GitHub Pages senza alcun server. Questa scelta garantiva semplicità di deploy e assenza di costi infrastrutturali, ma era incompatibile con le API

Routes, rendendo impossibile implementare funzionalità che richiedessero stato condiviso tra dispositivi diversi.

La versione corrente (**v2**) abbandona l'esportazione statica e adotta Vercel come piattaforma di hosting. Le API Routes di Next.js sono ora abilitate e vengono eseguite come funzioni serverless al momento della richiesta. Lo stato delle sessioni è persistito su Upstash Redis, un servizio di key-value store accessibile tramite API HTTP, compatibile con l'ambiente serverless di Vercel. La versione v1 rimane accessibile all'indirizzo storico su GitHub Pages (`https://samueldebenedictis.github.io/tesi/`); la versione v2 è disponibile all'indirizzo `https://www.imprevisti.app/`[1]. Il codice sorgente è pubblico su GitHub: `https://github.com/samueldebenedictis/tesi`.

4.3 Storyboard: l'uso dell'applicazione da parte dell'educatore

Questa sezione ripercorre passo per passo una sessione tipo dal punto di vista dell'educatore (o dello psicologo) che la conduce, dalla preparazione della partita alla chiusura. Per ogni fase si indicano che cosa fa l'educatore, che cosa fa l'applicazione e a quale scopo. Le singole schermate sono descritte nella sezione 4.4.

4.3.1 Fase 1 — Preparazione della partita

La preparazione è comune alle due modalità ed è il momento in cui l'educatore adatta il gioco al gruppo (figure 2 e 3, tabella 1).

Tabella 1: Fase 1: preparazione della partita.

N.	L'educatore	L'applicazione	Finalità
1	Sceglie la modalità: schermo singolo o multi-dispositivo.	Mostra la selezione; "Cambia modalità" permette di tornare indietro.	Adattare la sessione agli spazi e al gruppo (sezione 4.1).
2	Solo in schermo singolo: imposta numero di giocatori (2–10) e nomi. In multi-dispositivo i giocatori si registrano dallo smartphone.	Assegna un colore a ciascun giocatore.	Rendere ogni giocatore riconoscibile sul tabellone.

[1] Indirizzo alternativo su Vercel: `https://tesi-nine.vercel.app/`.

N.	L'educatore	L'applicazione	Finalità
3	Definisce la lunghezza del percorso (10–100 caselle).	Adegua la griglia del tabellone.	Regolare la durata della sessione.
4	Sceglie le tipologie di caselle speciali e la loro percentuale.	Genera il tabellone con le sole tipologie scelte.	Orientare la sessione verso le competenze da esercitare, o escludere sfide non adatte al gruppo.
5	Attiva o disattiva la Battaglia.	Se attiva, gestisce le collisioni tra pedine.	Dosare la componente competitiva.
6	Facoltativo: apre la modalità avanzata e assegna a mano il tipo di ogni casella (figura 13).	Sostituisce la distribuzione casuale.	Costruire un percorso su misura.
7	Preme "Inizia gioco".	Mescola i mazzi e salva la configurazione nel `localStorage`.	Avviare la partita; la configurazione resta disponibile dopo un ricaricamento.

4.3.2 Fase 2 — Sessione a schermo singolo

Tutti i partecipanti guardano lo stesso schermo. L'educatore conduce la partita e resta il riferimento per la valutazione delle sfide (figure 4 e 5, tabella 2).

Tabella 2: Fase 2: sessione a schermo singolo.

N.	L'educatore	L'applicazione	Finalità
1	Invita il giocatore di turno a premere il pulsante del dado.	Anima il lancio e sposta la pedina casella per casella.	Turni condivisi, senza dadi fisici.
2	Osserva la casella raggiunta; se è normale il turno passa al giocatore successivo.	Aggiorna barra laterale e turno.	—
3	Se la casella è speciale, conduce la sfida mostrata nella finestra modale (figure 8–11).	Estrae la carta dal mazzo corrispondente e la mostra a tutti.	Esercitare la competenza associata alla sfida (tabella 4).
4	Valuta l'esito e lo registra.	Applica le conseguenze: avanzamento, retrocessione o salto del turno.	L'educatore resta il giudice della sfida.

N.	L'educatore	L'applicazione	Finalità
5	Se due pedine occupano la stessa casella e la Battaglia è attiva, guida la sfida a morra cinese.	Attiva la Battaglia e applica l'esito sul tabellone.	Competizione regolamentata.
6	Alla partita conclusa proclama il vincitore.	Mostra la schermata di fine partita.	Gestire vittoria e sconfitta.

4.3.3 Fase 3 — Sessione multi-dispositivo

Ogni giocatore usa il proprio smartphone, mentre il tabellone resta sullo schermo dell'educatore (figure 6 e 7, tabella 3). Le scelte tecniche che stanno dietro a questi passi sono descritte nella sezione 4.8.

Tabella 3: Fase 3: sessione multi-dispositivo.

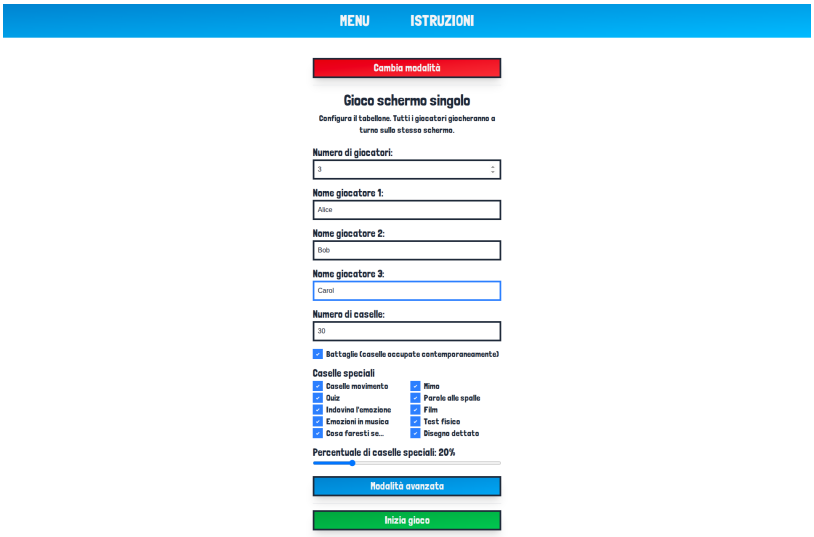
N.	L'educatore	L'applicazione	Finalità
1	Crea la sessione dalla configurazione e mostra il codice QR ai partecipanti.	Genera l'identificativo di sessione e il token dell'educatore; mostra la lobby.	Accesso dei giocatori senza digitare indirizzi.
2	Attende che i giocatori inquadrino il QR e inseriscano il proprio nome.	Assegna a ciascuno un identificativo e aggiorna la lobby.	Ogni partecipante gioca dal proprio dispositivo.
3	Preme "Avvia partita".	Porta tutti i dispositivi alla schermata di gioco.	—
4	Segue il tabellone; il giocatore di turno lancia il dado dal proprio smartphone.	Propaga il risultato a tutti; gli altri vedono chi è di turno.	Il tabellone resta il riferimento comune.
5	Sulle caselle speciali supervisiona la sfida: mostra la risposta (Quiz), indica chi ha indovinato (Mimo) o registra l'esito.	Distribuisce la sfida ai dispositivi, in forma condivisa o privata (tabella 4).	Preservare l'elemento di sorpresa nelle sfide private.
6	Conclude la partita.	Mostra il vincitore e il pulsante "Dai un feedback!".	Passare alla raccolta di feedback.

4.3.4 Fase 4 — Chiusura

Al termine, o in caso di interruzione, l'educatore può:



Passo 1: scelta della modalità.



Passi 2–5 e 7: configurazione (schermo singolo).

Figura 2: Preparazione della partita (1/2).



Passi 3–5 e 7: configurazione (multi-dispositivo).

Figura 3: Preparazione della partita (2/2).



Passo 1: lancio del dado.



Passi 3–4: sfida (Quiz) e registrazione dell'esito.

Figura 4: Sessione a schermo singolo (1/2).



Passo 4: turno saltato.



Passo 6: fine della partita.

Figura 5: Sessione a schermo singolo (2/2).



Passo 1: lobby con codice QR.



Passo 2: registrazione dal telefono.

Passo 4: giocatore di turno.

Figura 6: Sessione multi-dispositivo (1/2).



Passi 3–5: tabellone dell'educatore.





Passo 4: gli altri giocatori.



Passo 6: fine partita.

Figura 7: Sessione multi-dispositivo (2/2).

- riprendere la sessione in un secondo momento: la partita è conservata nel browser e può essere esportata come file JSON e ricaricata (sezione 4.4.5);
- invitare i partecipanti a compilare il modulo di feedback (sezione 5.1);
- consultare le risposte aggregate nella dashboard di analisi (sezione 4.4.7).

4.4 Le schermate dell'applicazione

Questa sezione descrive le singole schermate incontrate nello storyboard. Se non diversamente indicato, gli screenshot riportati si riferiscono alla modalità a schermo singolo; le schermate proprie della modalità multi-dispositivo sono descritte nella sezione 4.4.8. La navigazione avviene tramite un header persistente che espone due elementi: un link diretto alla schermata delle istruzioni e un pulsante Menu che apre una finestra modale con i collegamenti a Nuova partita, Continua partita, Carica partita e Feedback.

4.4.1 Schermata di gioco

È la schermata principale, composta da tre aree:

- **Tabellone** — Griglia di caselle numerate, identificate visivamente per tipologia; le pedine si animano al momento dello spostamento.
- **Barra laterale** — Mostra il giocatore attivo, il numero del round e le posizioni di tutti i giocatori.
- **Dado e modali** — Un pulsante permette al giocatore attivo di lanciare il dado; se la casella di arrivo è speciale, si apre la finestra modale corrispondente alla sfida.

Le sfide attivabili sono quelle descritte nella sezione 1.5. Ciascuna viene presentata attraverso una finestra modale sovrapposta al tabellone al momento dell'attivazione, come mostrato, per la modalità a schermo singolo, nelle figure 8–11; la tabella 4 riassume come è gestita nelle due modalità.

Nella modalità schermo singolo ogni sfida compare nella finestra modale visibile a tutti e l'educatore ne registra sempre l'esito. Nella modalità multi-dispositivo la visibilità dipende dalla sfida, come riassunto nella tabella 4.



Mimo.

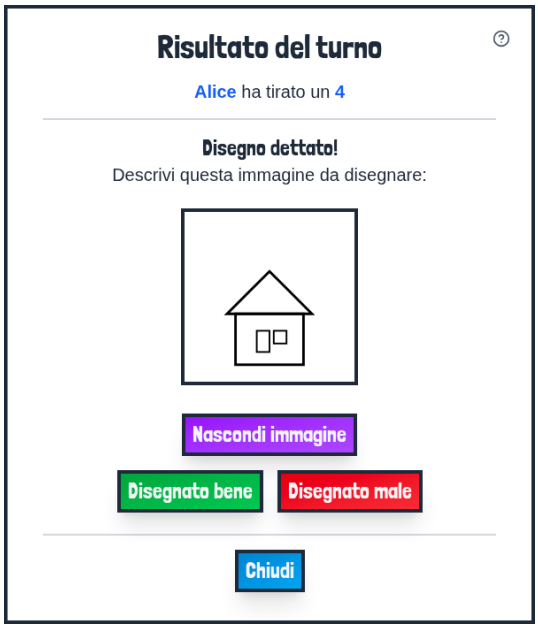


Quiz.

Figura 8: Finestre modali delle sfide, modalità a schermo singolo (1/4): Mimo, Quiz.



Parole alle spalle.

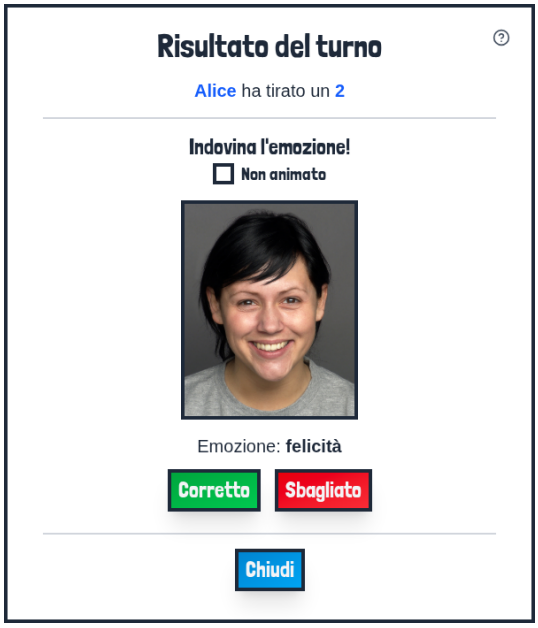


Disegno dettato.

Figura 9: Finestre modali delle sfide, modalità a schermo singolo (2/4): Parole alle spalle, Disegno dettato.



Emozioni in musica.



Indovina l'emozione.

Figura 10: Finestre modali delle sfide, modalità a schermo singolo (3/4): Emozioni in musica, Indovina l'emozione.



Cosa faresti se…



Test fisico.

Figura 11: Finestre modali delle sfide, modalità a schermo singolo (4/4): Cosa faresti se…, Test fisico.

Tabella 4: Gestione delle sfide nella modalità multi-dispositivo.

Sfida	Visibilità del contenuto	Esito
Quiz	Condivisa: la domanda è visibile a tutti, la risposta solo sul tabellone.	L'educatore mostra la risposta e registra successo o insuccesso.
Mimo	Privata: il tema compare solo sul telefono del giocatore attivo.	L'educatore indica quale giocatore ha indovinato, che riceve il beneficio.
Parole alle spalle, Disegno dettato	Privata: la parola o il disegno compaiono solo sul telefono del giocatore attivo, che sceglie il compagno destinatario.	Se la sfida riesce, entrambi avanzano di una casella.
Indovina l'emozione, Film, Emozioni in musica, Test fisico, Cosa faresti se…	Condivisa: visibile a tutti.	L'educatore registra l'esito al termine.
Battaglia	Nessun contenuto: la morra cinese si svolge dal vivo e sul tabellone compaiono solo i nomi dei due giocatori.	L'educatore indica il vincitore; il server aggiorna le posizioni.
Caselle movimento	Nessuna sfida.	Applicate in automatico.

4.4.2 Menu di navigazione

Il menu è accessibile in qualsiasi momento tramite il pulsante **MENU** nell'header, indipendentemente dalla schermata corrente. L'apertura sovrappone una finestra modale al contenuto sottostante senza interrompere lo stato della partita.

Il menu espone quattro collegamenti:

- **Continua partita** — reindirizza alla schermata di gioco, riprendendo la sessione in corso.
- **Nuova partita** — reindirizza alla schermata Home per configurare una partita da zero.
- **Carica partita** — reindirizza alla schermata Ripristina partita.
- **Dai un feedback!** — reindirizza alla schermata Feedback.

Una sezione dedicata alle impostazioni audio consente di abilitare o disabilitare, tramite due caselle di spunta indipendenti, gli **effetti sonori** e gli **annunci vocali** dell'applicazione; la modifica è immediata e persistente per tutta la sessione. Gli annunci vocali —

disattivati di default — leggono ad alta voce l'esito di ogni turno, il salto turno e la vittoria finale, offrendo un canale uditivo che rinforza le informazioni già mostrate a schermo: un supporto utile in particolare per chi fatica a seguire simultaneamente più stimoli visivi. Nella modalità multi-dispositivo gli annunci sono riprodotti dal tabellone condiviso.

4.4.3 Schermata Istruzioni

Presenta le regole del gioco strutturate in sezioni (introduzione, configurazione, svolgimento del turno, caselle speciali, fine della partita), navigabili tramite un indice interno con link di ancoraggio. Ogni sezione è corredata da video dimostrativi che mostrano l'applicazione in funzione nei principali scenari di gioco. La schermata è raggiungibile direttamente dall'header in qualsiasi momento, senza perdere lo stato della partita in corso.

4.4.4 Schermata Modalità avanzata

Consente di assegnare manualmente il tipo a ogni singola casella del tabellone, sostituendo la distribuzione casuale. L'interfaccia presenta un elenco scorrevole di tutte le caselle: ciascuna è identificata da un badge numerico il cui colore riflette il tipo attualmente assegnato. La prima e l'ultima casella (Start e Fine) sono bloccate e non modificabili. Per ogni casella intermedia è disponibile un menu a tendina con tutte le tipologie disponibili; selezionando il tipo *movimento* compare un campo numerico aggiuntivo per specificare il numero di caselle di avanzamento o retrocessione (da −5 a +5). I pulsanti in fondo alla schermata permettono di azzerare la configurazione, applicarla tornando alla schermata principale o annullare le modifiche.

4.4.5 Salvataggio e ripristino della partita

Lo stato completo della partita (posizioni delle pedine, turno corrente, stato dei mazzi) è serializzabile in formato JSON ed esportabile come file. La schermata di ripristino consente di riprendere una sessione interrotta attraverso due modalità distinte. La prima permette di caricare un file JSON esportato in precedenza: l'utente seleziona il file dal proprio dispositivo e avvia il ripristino; il sistema valida il contenuto e, in caso di file non valido, mostra un messaggio di errore. La seconda modalità è disponibile quando il browser ha conservato automaticamente una partita nel `localStorage`: in tal caso compare un pulsante aggiuntivo per riprendere la sessione senza bisogno di alcun file. In entrambi i casi, al termine del ripristino l'utente viene reindirizzato direttamente alla schermata di gioco.

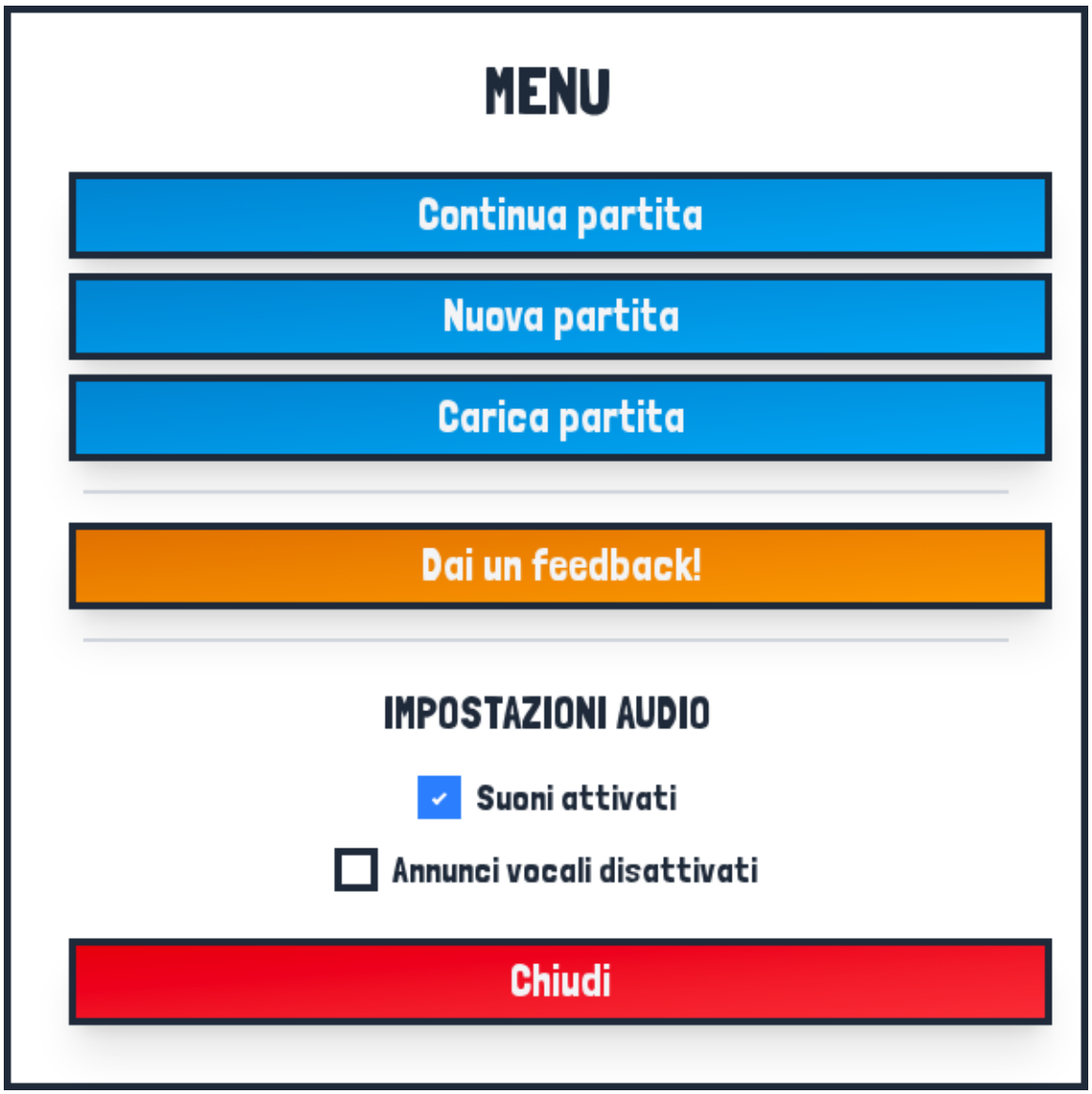


Figura 12: Menu di navigazione: collegamenti alle schermate principali e controllo audio.

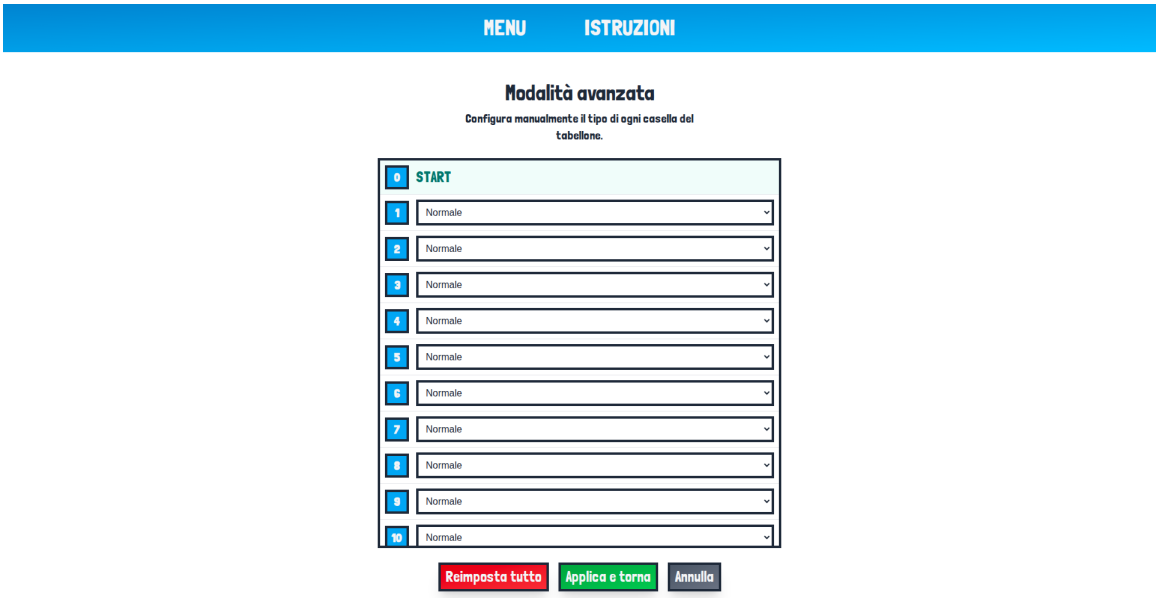


Figura 13: Schermata modalità avanzata: assegnazione manuale del tipo a ogni casella del tabellone.

4.4.6 Schermata Feedback

Il modulo, compilato al termine delle sessioni da docenti e alunni del CFPIL, raccoglie i dati di base del compilatore, il questionario standardizzato *System Usability Scale* (SUS) [8], alcune valutazioni numeriche e tre campi di testo libero; i campi sono elencati nella sezione 5.1. Le risposte, corredate dal numero di versione dell'applicazione al momento dell'invio, sono trasmesse all'endpoint interno `/api/feedback` e archiviate su Redis per analisi successive; un endpoint protetto da chiave segreta consente di esportare le risposte raccolte.

4.4.7 Dashboard di analisi dei feedback

La schermata `/admin/feedback`, protetta dalla stessa chiave segreta dell'endpoint di esportazione, presenta in forma sintetica le risposte raccolte dal modulo Feedback. La pagina interroga `/api/feedback` e calcola lato client il punteggio SUS medio secondo la formula standard [8], mostrandolo insieme a un grado qualitativo. Seguono le medie delle valutazioni numeriche e quattro grafici a barre con la distribuzione dei compilatori per ruolo, esperienza di gioco, identificazione neurodivergente e versione dell'applicazione: ciascuna barra è cliccabile e applica un filtro che ricalcola tutte le metriche della pagina sul sottoinsieme selezionato (i filtri sono cumulabili e rimovibili singolarmente). Il filtro

per versione permette, ad esempio, di isolare i feedback raccolti dopo un rilascio specifico. L'elenco dei commenti liberi chiude la pagina.

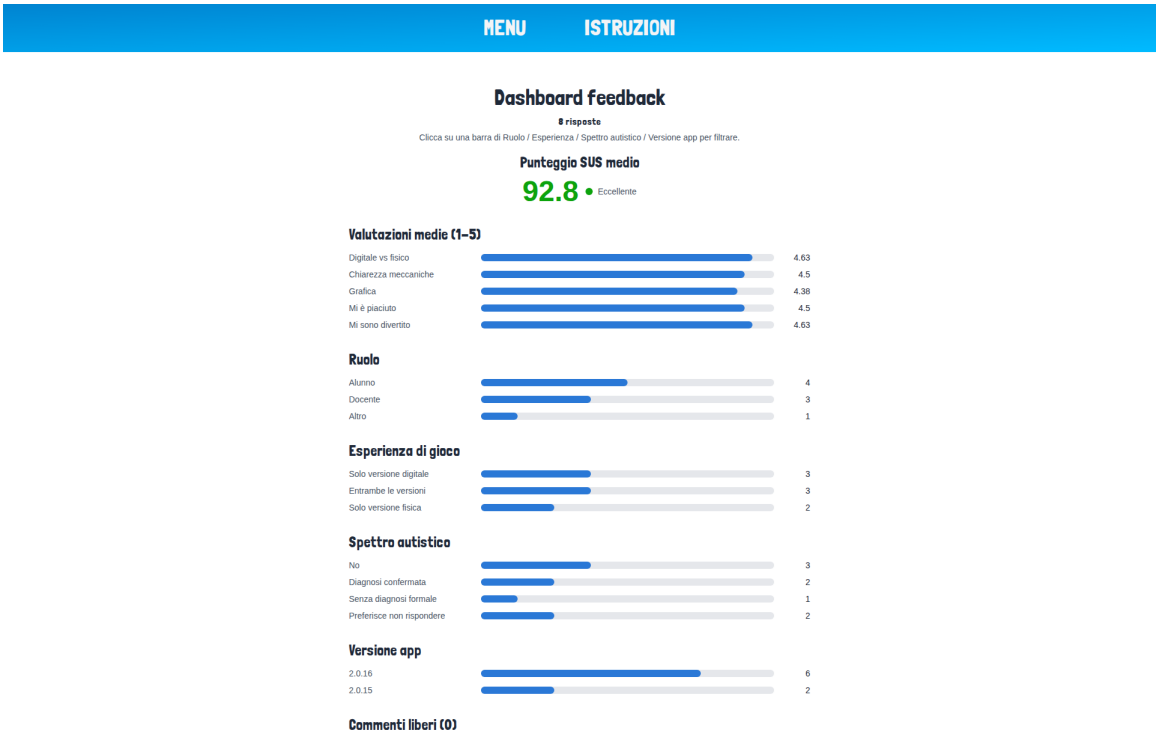


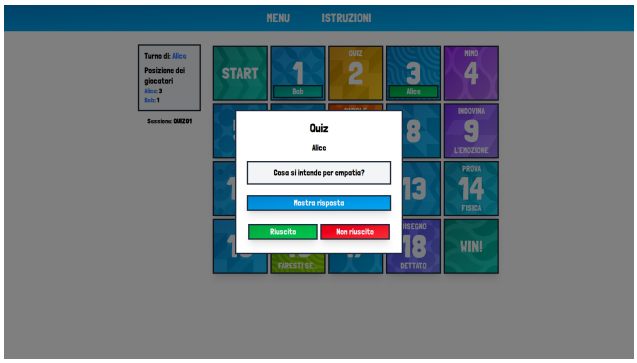
Figura 14: Dashboard di analisi dei feedback: punteggio SUS medio, valutazioni e distribuzione dei compilatori per categoria.

4.4.8 Modalità multi-dispositivo

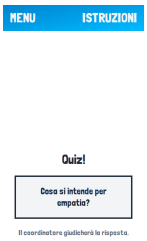
Le schermate specifiche di questa modalità — lobby con codice QR, pagina di registrazione e schermata personale del giocatore — compaiono nello storyboard (figure 6 e 7).

Le sfide speciali si comportano diversamente a seconda della tipologia: per alcune il contenuto è **condiviso** e compare identico sul tabellone e sullo smartphone del giocatore attivo; per altre è **privato** e il tema resta visibile solo sul dispositivo di chi deve eseguirlo, mentre l'host vede unicamente i nomi dei giocatori coinvolti, preservando l'elemento a sorpresa.

Per le sfide più dinamiche e fisiche (Mimo, Test fisico), la parte digitale si limita a distribuire il tema e a raccogliere l'esito: l'esecuzione vera e propria avviene fisicamente in aula ed è quindi meglio documentata con un video piuttosto che con uno screenshot statico.

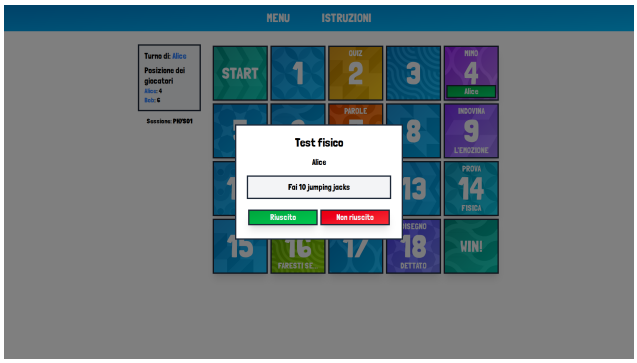


Tabellone dell'educatore: domanda visibile, con pulsante per rivelare la risposta corretta.



Smartphone del giocatore: stessa domanda, in attesa del giudizio dell'educatore.

Figura 15: Sfida condivisa (Quiz): la domanda compare identica su entrambi gli schermi, ma la risposta resta visibile solo sul tabellone.



Tabellone dell'educatore.



Smartphone del giocatore.

Figura 16: Sfida condivisa (Test fisico): il contenuto compare identico su entrambi gli schermi.

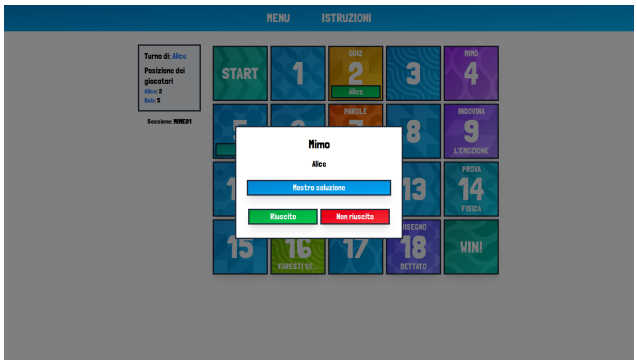


Tabellone dell'educatore: nessun dettaglio della sfida è visibile, solo i nomi dei giocatori coinvolti.

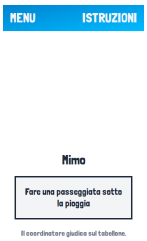


Smartphone del giocatore che descrive: tema e immagine di riferimento.

Figura 17: Sfida privata (Disegno dettato): il tema è visibile solo sul dispositivo del giocatore attivo.



Tabellone dell'educatore: il mimo da eseguire è nascosto, pronto per essere rivelato su richiesta.



Smartphone del giocatore che mima: la parola o frase da mimare.

Figura 18: Sfida privata (Mimo) nella modalità multi-dispositivo.

L'architettura tecnica che supporta questa modalità è descritta nella sezione 4.8.

4.5 Architettura dei moduli

Nella modalità schermo singolo, l'applicazione segue un'architettura a tre strati con separazione netta tra logica di dominio, gestione dello stato e presentazione, interamente eseguita nel browser. La modalità multi-dispositivo, descritta nella sezione 4.8, sposta l'esecuzione autoritativa del Domain Model lato server e riduce Zustand, sul lato client, a un ruolo più marginale:

Strato	**Modulo**
Presentation Layer	Componenti React (`src/app/`)
State Management Layer	Store Zustand (`src/store/`)
Domain Model Layer	Logica di gioco (`src/model/`)

Tabella 5: Architettura a strati dell'applicazione (modalità schermo singolo).

Questa separazione consente di testare la logica di gioco indipendentemente dall'interfaccia e di aggiornare i componenti visivi senza modificare le regole di gioco.

4.5.1 Domain Model

Il domain model contiene tutta la logica di gioco, completamente slegata da React e dal browser. La classe `Game` è l'orchestratore principale: mantiene i riferimenti a tutti i sottocomponenti (tabellone, giocatori, dado, mazzi, manager) ed espone i metodi pubblici che il layer di stato chiama per far avanzare la partita. Il metodo `playTurn()` esegue un turno completo — lancio del dado, spostamento della pedina, elaborazione della casella — e restituisce il tipo di azione da mostrare all'utente. I metodi `toJSON()` e `fromJSON()` gestiscono la serializzazione dello stato per il salvataggio su file.

Ogni aspetto della partita è delegato a un manager specializzato:

- `TurnManager` — traccia il turno corrente, il round e il giocatore attivo;
- `MovementManager` — sposta le pedine, gestisce bonus/malus di movimento e collisioni;
- `GameStateManager` — verifica la condizione di vittoria;
- `SpecialSquareProcessor` — determina quale azione speciale si attiva su una casella;

- `BattleManager` e i manager di ogni tipologia di sfida (es. `MimeManager`, `QuizManager`) — applicano gli effetti dell'esito di ciascuna sfida.

Le caselle del tabellone sono modellate come classi distinte (es. `MimeSquare`, `QuizSquare`) che estendono la classe base `Square` e sono ricostruite tramite una *factory* al momento del ripristino di una partita salvata.

Ogni tipologia di casella speciale dispone di un mazzo di carte dedicato (es. `MimeDeck`, `QuizDeck`), mescolato all'inizio della partita e consumato in sequenza per garantire varietà e non ripetizione dei contenuti nella stessa sessione.

Il mazzo della sfida "Indovina l'emozione" (`FaceEmotionDeck`) utilizza fotografie tratte da FACES, un database di volti che esprimono emozioni di base (felicità, rabbia, tristezza, paura, disgusto, neutralità), sviluppato dal Max Planck Institute for Human Development e concesso gratuitamente per la ricerca scientifica [9]. Sono utilizzati i sei soggetti del sottoinsieme pubblico (004, 066, 079, 116, 140, 168), gli unici le cui fotografie possono essere mostrate per illustrare la metodologia di ricerca. A differenza delle illustrazioni disegnate usate per le altre sfide, l'impiego di volti fotografici reali e validati scientificamente rende più realistico e trasferibile l'esercizio di riconoscimento delle espressioni facciali.

Per ciascuna espressione non neutra è disponibile anche un'animazione di transizione (*face morphing*) dall'espressione neutra a quella emotiva, generata automaticamente a partire dalle coppie di fotografie dello stesso soggetto. Uno script Python (`scripts/face_morph.py`, eseguibile con `make face-morph`) rileva i 478 punti di riferimento del volto con il modello *Face Landmarker* di MediaPipe, li triangola con la triangolazione di Delaunay e deforma ciascun triangolo in modo progressivo, miscelando i due scatti con una dissolvenza incrociata. La trasformazione è applicata solo all'interno di una maschera sfumata attorno al volto: sfondo e vestiario si spostano leggermente da uno scatto all'altro e, se deformati, mostrerebbero artefatti evidenti. Il risultato è un insieme di 60 gif (cinque espressioni, sei soggetti, due versioni di scatto), nominate come le fotografie di origine con il suffisso `_morph`. Nell'interfaccia l'uso dell'animazione è una scelta esplicita, tramite l'opzione "Animato"/"Non animato" (disattivata di default), memorizzata in `localStorage` da uno store Zustand dedicato (`useFaceEmotionStore`). Il componente `FaceEmotionImage` che la gestisce è condiviso dalla finestra modale a schermo singolo, dall'*overlay* dell'educatore e dalla schermata del giocatore di turno; se la gif non è disponibile, ripiega sulla fotografia statica.

La generazione procede, per ciascuna coppia (fotografia neutra, fotografia con espressione) dello stesso soggetto e della stessa versione di scatto, nei passi seguenti:

1. le due fotografie, di circa 3500 px di larghezza, sono ridimensionate a 480 px di larghezza per contenere il peso della gif;

2. su ciascuna sono rilevati i 478 punti di riferimento, ai quali si aggiungono otto punti fissi sul bordo dell'immagine (angoli e punti medi dei lati), in modo che la triangolazione copra l'intero fotogramma;

3. la triangolazione di Delaunay è calcolata sulla media delle posizioni dei punti nelle due fotografie e resta la stessa per tutti i fotogrammi;

4. per ogni fotogramma intermedio, con un parametro di avanzamento α tra 0 e 1, ciascun punto è portato nella posizione interpolata linearmente tra le due fotografie, i triangoli di entrambe le immagini sono deformati con una trasformazione affine verso tale posizione e i due risultati sono miscelati con pesi $1 - \alpha$ e α; α segue una curva *ease-in/ease-out*, così che la transizione acceleri e rallenti gradualmente;

5. il fotogramma finale è composto dentro la maschera del volto (inviluppo convesso dei punti del viso, sfumato con un filtro gaussiano con nucleo di 31 px), mentre l'esterno è preso invariato dalla fotografia neutra.

La sequenza comprende 21 fotogrammi di transizione, una pausa di 6 fotogrammi sul volto neutro e sull'espressione, e il percorso inverso fino al neutro; la riproduzione è a 15 fotogrammi al secondo, in ciclo continuo.

Il mazzo della sfida "Film" (`FilmDeck`) contiene cinque scene, una per ciascuna emozione di base della sfida precedente esclusa la neutralità (felicità, rabbia, tristezza, disgusto, paura). La casella segue la stessa struttura delle altre sfide di riconoscimento delle emozioni: una `FilmSquare` esegue un comando che pesca una carta dal mazzo e restituisce un oggetto `Film`, la cui risoluzione è affidata a un `FilmManager`. Ogni carta associa l'emozione corretta a un file video servito dall'applicazione (cartella `public/videos`), in formato H.264 con audio AAC, risoluzione 852×480 e durata compresa tra circa 17 e 24 secondi. Poiché il mazzo ha solo cinque carte e viene rimescolato una volta esaurito, nelle partite lunghe le scene si ripetono.

A differenza delle fotografie, la scena è un contenuto con voce e musica. Il componente `FilmVideo` usa il lettore nativo del browser, con i comandi visibili e senza avvio automatico, così che la riproduzione parta solo per scelta dell'utente e possa essere ripetuta; non sono disponibili sottotitoli. Il giocatore guarda la scena e indica l'emozione; l'educatore mostra la risposta con il pulsante "Mostra risposta" e registra l'esito. Se la risposta è corretta il giocatore avanza di una casella, altrimenti salta il turno successivo, come in "Indovina l'emozione".

Le scene erano tratte da film commerciali, per i quali non si dispone dei diritti di utilizzo: nella versione attuale la sfida è quindi disattivata e potrà essere riattivata con scene dotate di licenza adeguata.

4.5.2 State Management

Lo stato globale è gestito con Zustand, suddiviso in quattro store indipendenti persistiti in `localStorage`:

- `game-store` — stato della partita in corso: istanza serializzata di `Game`, risultato del dado, tipo di azione corrente, stato delle finestre modali;
- `config-store` — configurazione della partita: nomi e numero dei giocatori, numero di caselle, tipologie e percentuale di caselle speciali, attivazione della Battaglia;
- `sound-store` — preferenze dell'utente riguardo agli effetti sonori e agli annunci vocali;
- `face-emotion-store` — preferenza per la versione animata o statica dei volti di "Indovina l'emozione".

Questa descrizione vale per intero nella modalità schermo singolo, dove `game-store` è l'unica fonte di verità e le sue azioni (`playTurn()`, `resolve*()`) sono l'unico punto d'ingresso per far avanzare la partita. Nella modalità multi-dispositivo lo stato autoritativo risiede invece sul server (sezione 4.8.3): sulla pagina host `game-store` viene reidratato in sola lettura dallo snapshot ricevuto via *polling* (`Game.fromJSON(session.gameState)`) e serve solo a renderizzare il tabellone, mentre le azioni dell'utente — avvio partita, risoluzione delle sfide, lancio del dado — vengono inviate direttamente alle API Routes tramite `fetch()`, bypassando le azioni dello store. Le pagine del giocatore (`app/player/`) e di accesso (`app/join/`) non usano Zustand: si affidano solo a `useState` locale e all'hook di *polling* `useSessionPolling`.

4.5.3 Presentation Layer

Il presentation layer comprende sia le pagine che i componenti React. In Next.js le pagine (`page.tsx`) sono esse stesse componenti: la distinzione è strutturale — determinano il routing — ma non concettuale.

Le pagine dell'applicazione sono:

- `app/page.tsx` — schermata di selezione modalità e configurazione della partita;
- `app/game/page.tsx` — schermata principale di gioco (modalità schermo singolo);
- `app/multiplayer/page.tsx` — lobby per la modalità multi-dispositivo (creazione sessione e QR code);
- `app/multiplayer/[sessionId]/page.tsx` — tabellone host per la sessione multi-dispositivo;
- `app/player/[sessionId]/[playerId]/page.tsx` — schermata del giocatore sullo smartphone;

- `app/join/[sessionId]/page.tsx` — pagina di accesso tramite QR code, reindirizza alla registrazione;
- `app/instructions/page.tsx` — istruzioni di gioco con video dimostrativi;
- `app/advanced-mode/page.tsx` — configurazione avanzata del tabellone;
- `app/restore-game/page.tsx` — ripristino partita da file;
- `app/feedback/page.tsx` — modulo di feedback;
- `app/admin/feedback/page.tsx` — dashboard di analisi dei feedback raccolti.

I componenti principali dell'interfaccia sono:

- `board.tsx` — tabellone come griglia CSS con numero di colonne variabile in base alla dimensione;
- `square.tsx` — singola casella con numero, tipo visivo e pedine presenti;
- `pawn.tsx` — pedina con colore assegnato e animazione al momento dello spostamento;
- `dice.tsx` — dado con animazione al momento del lancio;
- `left-bar.tsx` — barra laterale con turno attivo, round corrente e classifica;
- `turn-result-modal.tsx` — finestra modale delegata a un sottocomponente specifico per ogni tipo di sfida.

I componenti primitivi riutilizzabili (`Button`, `Input`, `Label`, `Select`) sono sviluppati e documentati in isolamento tramite Storybook.

4.6 Scelte implementative rilevanti

4.6.1 Il tipo `GameActionResult` e il Command Pattern

La comunicazione tra il domain model e il layer di stato avviene tramite il tipo `GameActionResult`, un oggetto TypeScript che `playTurn()` restituisce al termine di ogni turno. Questa scelta riflette il *Command Pattern*: invece di eseguire direttamente un'azione sull'interfaccia, il domain model produce un oggetto che *descrive* l'azione da compiere, lasciando al chiamante la responsabilità di decidere come gestirla.

Il campo `type` è un'unione discriminata di stringhe letterali — una per ogni tipologia di sfida più il valore `"none"` per i turni senza azione speciale. Il campo `data` trasporta i dati

specifici dell'azione (ad esempio la carta estratta o i giocatori coinvolti in una battaglia). Lo store legge `type` per determinare quale finestra modale aprire, e `data` per popolarla con il contenuto corretto, senza che il domain model abbia alcuna dipendenza dall'interfaccia. Il ciclo si chiude quando l'educatore registra l'esito della sfida: lo store chiama il corrispondente metodo `resolve*()` su `Game`, che applica le conseguenze sul tabellone e aggiorna lo stato della partita.

Listing 4.1: Tipo di ritorno di `playTurn()` (src/model/managers/types.ts)

```
export type GameActionResult = {
  type:
    | "battle" | "mime"    | "quiz"
    | "backwrite"          | "face-emotion"
    | "film"               | "music-emotion"
    | "physical-test"      | "what-would-you-do"
    | "dictation-draw"     | "none";
  data?: Battle | Mime | Quiz | BackWrite | FaceEmotion
       | Film | MusicEmotion | PhysicalTest
       | WhatWouldYouDo | DictationDraw;
  diceResult: number;
  actionType: string | null;
};
```

Il metodo `playTurn()` in `Game` sfrutta questo tipo per incapsulare l'intera logica del turno — gestione del turno saltato, lancio del dado, movimento, rilevamento collisioni ed elaborazione della casella speciale — restituendo un valore strutturato che il layer di stato consuma senza dover conoscere i dettagli interni.

Listing 4.2: Metodo `playTurn()` (src/model/game.ts)

```
playTurn(): GameActionResult {
  const currentPlayer = this.turnManager.getCurrentPlayer();

  if (currentPlayer.mustSkipTurn()) {
    this.turnManager.advanceTurn();
    return { type: "none", diceResult: 0, actionType: null };
  }

  const diceValue = this.dice.roll();
  const result = this.executePlayerTurn(currentPlayer,
     diceValue);
  this.turnManager.advanceTurn();

  return { ...result, diceResult: diceValue };
}
```

4.6.2 Gestione dello stato con Zustand e persistenza

Quanto segue descrive il funzionamento di `useGameStore` nella modalità schermo singolo, dove è l'unico punto di raccordo tra il domain model e l'interfaccia utente; il suo ruolo ridotto nella modalità multi-dispositivo è descritto nella sezione 4.8.3. È definito tramite `create()` di Zustand, avvolto nel middleware `persist`, e si articola in due sezioni distinte: lo *stato* e le *azioni*.

La sezione di stato contiene due categorie di campi: quelli che riflettono la partita in corso (`game`, istanza della classe `Game`, e `gameData`, la sua rappresentazione JSON) e quelli che descrivono la situazione dell'interfaccia (`isModalOpen`, `isDiceModalOpen`, `diceResult`, `actionType`, `actionData` e altri).

Le azioni sono funzioni che i componenti React chiamano in risposta agli eventi dell'utente. La più importante è `playTurn()`: recupera l'istanza di `Game` dallo store, invoca il metodo omonimo del domain model, riceve il `GameActionResult` e aggiorna lo stato dell'interfaccia di conseguenza — impostando il risultato del dado, il tipo di azione attiva e aprendo la finestra modale appropriata. Analogamente, ciascuna azione `resolve*()` delega la risoluzione al metodo corrispondente di `Game` e, al termine, aggiorna la serializzazione JSON della partita tramite `toJSON()`.

Il middleware `persist` sincronizza automaticamente lo store con il `localStorage` del browser. L'opzione `partialize` limita la persistenza al solo campo `gameData`: in questo modo lo stato della partita sopravvive al ricaricamento della pagina, mentre i campi transitori dell'interfaccia vengono reinizializzati ai valori di default.

Listing 4.3: Store di gioco con persistenza selettiva (src/store/game-store.ts)

```
export const useGameStore = create<GameStore>()(
  persist(
    (set, get) => ({
      game: null,
      gameData: null,
      isModalOpen: false,
      // ...altri campi UI...

      actions: {
        playTurn: () => {
          const { game } = get();
          if (!game) return;

          const { diceResult, actionType, data } =
             game.playTurn();
          // aggiorna lo stato UI in base al risultato...
        },
        // ...altri metodi...
```

```
18        },
19      }),
20      {
21        name: "game-store",
22        partialize: (state) => ({
23          gameData: state.gameData,
24          hasSavedGame: !!state.gameData,
25        }),
26      },
27    ),
28  );
```

4.7 Design pattern utilizzati

La progettazione dell'applicazione ha fatto uso di diversi design pattern consolidati [10], applicati in modo coerente con le responsabilità dei moduli descritti nelle sezioni precedenti.

4.7.1 Command Pattern

Il pattern Command è applicato nella comunicazione tra il domain model e il layer di stato tramite il tipo `GameActionResult` (si veda la sezione 4.6.1): ogni turno produce un oggetto che *descrive* l'azione da eseguire, disaccoppiando il mittente (il domain model) dal ricevente (lo store e l'interfaccia).

4.7.2 Facade Pattern

La classe `Game` agisce da *Facade*: espone un'interfaccia semplice e coerente (`playTurn()`, `resolve*()`, `toJSON()`) nascondendo la complessità interna dei manager, dei mazzi e del tabellone. I componenti React e lo store non interagiscono mai direttamente con i sottosistemi interni.

4.7.3 Factory Pattern

Il modulo `square-builder.ts` espone la funzione `squareFromJSON()`, una *factory* che ricostruisce le istanze delle caselle a partire dalla loro rappresentazione JSON durante il ripristino di una partita salvata. Poiché ogni tipologia di casella è una classe distinta, la factory centralizza la logica di selezione del costruttore corretto in base al campo `type`.

4.7.4 Observer Pattern

Zustand adotta implicitamente il pattern Observer: i componenti React si *sottoscrivono* a porzioni specifiche dello store tramite selettori, e vengono ri-renderizzati automaticamente solo quando i campi osservati cambiano. Questo evita aggiornamenti superflui dell'interfaccia e mantiene i componenti disaccoppiati dalla logica di aggiornamento dello stato.

4.7.5 Manager Pattern

La classe `Game` delega ciascuna responsabilità a un manager specializzato (sezione 4.5.1). Questo approccio — talvolta chiamato *Manager Pattern* o *Specialist Pattern* — consente di mantenere la classe `Game` snella e leggibile, concentrando ciascuna regola di gioco nel modulo che ne è responsabile, con benefici diretti sulla testabilità e sulla manutenibilità del codice.

4.7.6 Model-View-Controller

Nella modalità schermo singolo, l'architettura a tre strati dell'applicazione rispecchia il pattern architetturale *Model-View-Controller* (MVC): il Domain Model (`src/model/`) corrisponde al *Model*, i componenti React (`src/app/`) alla *View* e gli store Zustand (`src/store/`) al *Controller*, che media le interazioni dell'utente verso il modello e propaga gli aggiornamenti di stato verso l'interfaccia. Nella modalità multi-dispositivo il ruolo di Controller passa alle API Routes, che ricevono le azioni via HTTP e le applicano a un'istanza di `Game` lato server (sezione 4.8.3); Zustand vi sopravvive solo come cache di rendering sulla pagina host.

4.8 Modalità multi-dispositivo

Di seguito si descrive l'architettura tecnica che supporta la modalità multi-dispositivo.

4.8.1 Architettura delle sessioni

La modalità multi-dispositivo richiede una componente *server-side* per sincronizzare lo stato della partita tra i dispositivi dei giocatori. A tal fine, l'applicazione fa uso delle *API Routes* di Next.js, che consentono di definire endpoint HTTP direttamente nel progetto senza la necessità di un server separato. La figura 19 riassume i componenti coinvolti e le comunicazioni descritte nel resto della sezione.

Lo stato di sessione è rappresentato da un oggetto `SessionState` che include l'elenco dei giocatori connessi, la fase corrente della partita, il turno attivo e gli esiti delle sfide. La

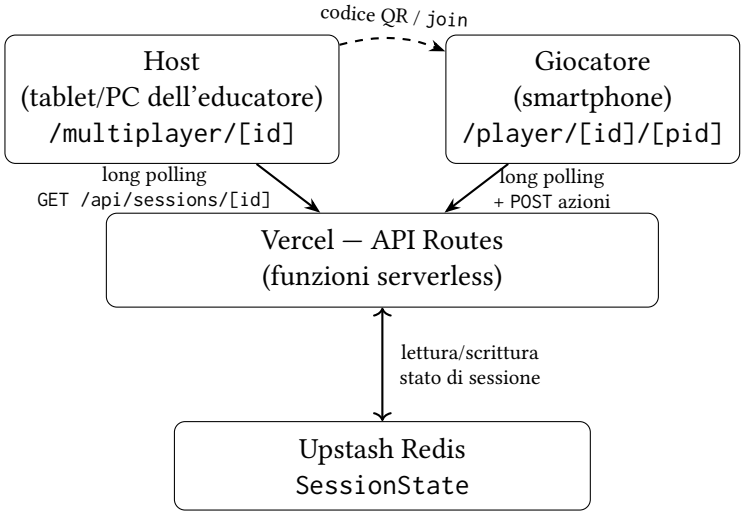


Figura 19: Architettura della modalità multi-dispositivo: host e giocatori comunicano con le API Routes tramite *long polling*, che leggono e scrivono lo stato di sessione su Redis; il giocatore si unisce inquadrando il codice QR mostrato dall'host.

persistenza di questo stato è gestita tramite un meccanismo a tre livelli, scelto in cascata in base all'ambiente di esecuzione:

1. **Upstash Redis** — in produzione, lo stato è memorizzato su Upstash Redis (descritto nel capitolo 3), tramite il client `@upstash/redis` configurato con le variabili di ambiente della piattaforma;

2. **ioredis** — in sviluppo locale con Redis disponibile, si utilizza il client `ioredis` per connettersi a un'istanza Redis locale;

3. **Map in memoria** — in assenza di Redis, lo stato è mantenuto in una `Map` JavaScript in memoria del processo Node.js, sufficiente per lo sviluppo locale su singola istanza.

Questa struttura garantisce portabilità tra ambienti senza modifiche al codice applicativo: la selezione del livello appropriato avviene in fase di inizializzazione in base alle variabili di ambiente disponibili.

La creazione di una sessione avviene tramite una richiesta `POST` all'endpoint `/api/sessions`, che restituisce un identificatore di sessione (`sessionId`) e un `hostToken`, un token opaco conservato nel `localStorage` del dispositivo dell'educatore e allegato alle successive richieste privilegiate per autorizzarle.

4.8.2 Connessione dei giocatori tramite QR code

Una volta creata la sessione, l'applicazione genera un codice QR che i giocatori inquadrano con il proprio smartphone. Il codice QR contiene l'URL della pagina di accesso, che include il `sessionId` come parametro. Il giocatore inserisce il proprio nome e invia una richiesta `POST` all'endpoint `/api/sessions/[id]/join`, che registra il nuovo partecipante nella sessione e restituisce un `playerId` univoco.

Il giocatore viene quindi reindirizzato alla propria schermata di gioco, raggiungibile all'URL `/player/[sessionId]/[playerId]`. L'identità del giocatore è interamente codificata nell'URL: anche in caso di chiusura accidentale della scheda o di ricaricamento della pagina, il giocatore recupera il proprio stato semplicemente riaprendo lo stesso indirizzo.

4.8.3 Stato lato client: il ruolo ridotto di Zustand

A differenza della modalità schermo singolo (sezione 4.5.1), in modalità multi-dispositivo l'istanza di `Game` autoritativa vive lato server: le API Routes la ricostruiscono da `SessionState` tramite `Game.fromJSON()`, invocano su di essa i metodi del domain model e ne persistono il risultato su Redis. Il client non possiede quindi una copia mutabile della partita, ma solo lo snapshot pubblico ricevuto tramite *long polling*.

Questo capovolge il ruolo di Zustand rispetto alla modalità schermo singolo:

- sulla pagina host (`app/multiplayer/[sessionId]/page.tsx`), `game-store` viene reidratato in sola lettura a ogni aggiornamento del *polling* (`gameStore.setGame(Game.fromJSON(session.gameState))`) e serve esclusivamente a renderizzare il tabellone; le azioni dell'educatore (avvio partita, risoluzione di una sfida, designazione di un vincitore) non passano per le azioni dello store ma vengono inviate come richieste `POST` dirette agli endpoint `/api/sessions/[id]/*`;
- le pagine `app/player/[sessionId]/[playerId]/page.tsx` e `app/join/[sessionId]/page.tsx` non importano né `useGameStore` né `useConfigStore`: si basano unicamente su `useState` locale e sull'hook `useSessionPolling` (`src/lib/use-session-polling.ts`), che espone direttamente lo stato pubblico di sessione ricevuto dal server.

In sintesi, la tabella 5 e la lettura MVC della sezione 4.7.6 descrivono l'architettura della sola modalità schermo singolo; in modalità multi-dispositivo lo strato di gestione dello stato — e con esso il ruolo di Controller — si sposta alle API Routes lato server, mentre Zustand resta attivo, ridotto a cache di presentazione, solo sulla pagina host.

4.8.4 Comunicazione in tempo reale tramite *long polling*

Per notificare i dispositivi dei giocatori degli aggiornamenti di stato senza l'uso di connessioni persistenti, la modalità multi-dispositivo adotta la tecnica del *long polling*. Ogni

dispositivo invia una richiesta `GET` all'endpoint `/api/sessions/[id]`, passando come parametro il valore `updatedAt` dell'ultimo stato ricevuto. Il server, anziché rispondere immediatamente, mantiene la connessione aperta per un massimo di otto secondi, interrogando lo *store* ogni trecento millisecondi; restituisce la risposta non appena rileva che il campo `updatedAt` dello stato corrente è più recente di quello del client, oppure allo scadere del *timeout*. Ricevuta la risposta, il client invia subito la richiesta successiva.

Questo approccio riduce il numero di invocazioni delle *serverless function* di circa cinque volte rispetto al *polling* a intervallo fisso, un aspetto rilevante ai fini della compatibilità con il piano gratuito di Vercel, che impone limiti al numero di esecuzioni mensili. La scelta del *long polling* in luogo dei WebSocket dipende dalla piattaforma: le *serverless function* di Vercel non supportano connessioni WebSocket persistenti, mentre le richieste HTTP di lunga durata sono gestite nativamente.

4.8.5 Rilevamento dell'inattività

Per evitare il consumo inutile di risorse in caso di dispositivi lasciati incustoditi, la modalità multi-dispositivo include un meccanismo di rilevamento dell'inattività. Dopo trenta minuti di inattività del giocatore sullo schermo compare un *overlay* con il messaggio *"Stai ancora giocando?"*, che invita l'utente a confermare la propria presenza; in questa fase il *polling* prosegue. Se l'*overlay* resta senza risposta per ulteriori cinque minuti, il *polling* viene interrotto. Qualsiasi interazione con il dispositivo — un tocco sullo schermo, un'azione di gioco — chiude l'*overlay* e riavvia il ciclo di aggiornamento.

4.8.6 Svolgimento del turno in modalità multi-dispositivo

Nel corso della partita, il flusso di ogni turno si svolge come segue:

1. L'educatore avvia la partita premendo il pulsante **Avvia partita** dalla schermata di *lobby*; tutti i dispositivi ricevono l'aggiornamento di stato e passano alla schermata di gioco.

2. Quando è il proprio turno, il giocatore attivo lancia il dado dal proprio dispositivo; il risultato viene comunicato al server e propagato agli altri partecipanti. Sul proprio schermo, il giocatore attivo visualizza prima una schermata intermedia con il valore del dado e la casella raggiunta, e prosegue premendo il pulsante **Avanti** prima di accedere alla sfida o al turno successivo.

3. Se la casella di arrivo non attiva alcuna sfida speciale, il turno termina e il controllo passa al giocatore successivo.

4. Se la casella attiva una sfida, la gestione varia in base alla tipologia (tabella 4). Mentre la sfida è in corso, i giocatori non coinvolti visualizzano sul proprio dispositivo

un'animazione (un'icona a forma di dado in rotazione) al posto di un semplice messaggio testuale, a indicare l'attesa dello svolgimento dell'azione da parte di un altro giocatore.

5. Al termine di ogni sfida, l'educatore conferma l'esito sull'interfaccia; il server aggiorna lo stato della sessione e tutti i dispositivi ricevono la notifica dell'avanzamento.

Al termine della partita, tutti i dispositivi visualizzano la schermata di fine gioco con il nome del vincitore; sui dispositivi dei singoli giocatori la schermata include anche il pulsante **Dai un feedback!**, che reindirizza alla schermata Feedback.

4.8.7 Considerazioni sulla dimensione sociale

L'introduzione della modalità multi-dispositivo modifica in modo rilevante la dinamica relazionale del gioco. Nella modalità schermo singolo tutti i partecipanti guardano lo stesso schermo: l'attenzione del gruppo converge su un punto fisico condiviso, favorendo quella che la letteratura indica come *joint attention* — un costrutto rilevante nell'intervento sull'ASD [3]. Le sfide si svolgono davanti a tutti, e l'interazione tra i giocatori è continua e visibile.

Nella modalità multi-dispositivo, invece, ciascun giocatore interagisce prevalentemente con il proprio schermo. Sebbene alcune sfide — come Mimo e Test fisico — mantengano una componente di interazione fisica diretta, l'attenzione tende a distribuirsi sui singoli dispositivi anziché convergere su un punto comune. Questa frammentazione potrebbe limitare, almeno parzialmente, la qualità dell'interazione faccia a faccia che costituisce uno dei vantaggi chiave del gioco da tavolo rispetto ai serious games individuali descritti dalla letteratura [5, 7].

Il vantaggio specifico della modalità multi-dispositivo risiede nell'accessibilità e nella gestione della segretezza: consente di giocare in contesti in cui non è disponibile uno schermo condiviso sufficientemente grande, e permette che per alcune sfide — come Mimo e Disegno dettato — il tema da rappresentare compaia esclusivamente sullo schermo del giocatore attivo, senza essere visibile agli altri prima che la sfida abbia inizio. La scelta tra le due modalità dipende pertanto dagli obiettivi dell'educatore: la modalità schermo singolo è preferibile quando l'obiettivo primario è stimolare la *joint attention* e l'interazione faccia a faccia; la modalità multi-dispositivo è indicata quando si intende promuovere l'uso autonomo della tecnologia o quando non è disponibile un unico schermo condiviso di dimensioni adeguate.

Capitolo 5

Test

5.1 Raccolta di feedback

Al termine delle sessioni di utilizzo, i partecipanti del CFPIL di Varese sono stati invitati a compilare il modulo di feedback integrato nell'applicazione. Per ridurre il carico cognitivo delle numerose domande, il modulo è organizzato come un percorso guidato a step (*wizard*) suddiviso in quattro fasi — informazioni di base, questionario SUS, valutazioni e domande aperte — con avanzamento vincolato alla compilazione dei campi obbligatori di ciascuna fase. Il modulo raccoglie:

- **nome**;
- **ruolo** — docente, alunno o altro;
- **esperienza pregressa con il gioco** — versione fisica, digitale o entrambe;
- **modalità di gioco** — schermo singolo o multischermo;
- **questionario SUS** — 10 affermazioni standardizzate della *System Usability Scale* [8], ciascuna valutata in scala 1–5;
- **fedeltà della versione digitale rispetto a quella fisica** — valutazione in scala 1–5, facoltativa;
- **chiarezza dei meccanismi di gioco** — valutazione in scala 1–5;
- **grafica** — valutazione in scala 1–5;
- **gradimento complessivo** — valutazione in scala 1–5;
- **divertimento** — valutazione in scala 1–5;

- **aspetti positivi**;
- **aspetti negativi**;
- **suggerimenti di miglioramento**;
- **identificazione neurodivergente/autistica** — facoltativa, utile per leggere i feedback anche in relazione a eventuali bisogni specifici;
- **versione dell'applicazione** — registrata automaticamente al momento dell'invio.

Le risposte raccolte sono state analizzate tramite la dashboard descritta nella sezione 4.4.7 (figura 14), che calcola il punteggio SUS medio e la distribuzione delle valutazioni per categoria senza richiedere l'esportazione manuale dei dati.

5.2 Risultati preliminari

Le risposte raccolte con la versione 2.0.8 dell'applicazione, in una prova svolta in modalità multi-dispositivo, provengono da partecipanti con ruolo "alunno" e "altro", senza docenti, tutti con esperienza della sola versione digitale. I nomi riportati nel modulo sono stati scelti dai compilatori in modo da garantirne la riservatezza. Trattandosi di un campione ridotto, i valori che seguono hanno valore puramente indicativo.

Misura	**Valore**
Punteggio SUS medio (scala 0–100)	90,0
Deviazione standard SUS	4,6
Punteggio SUS minimo–massimo	85,0–95,0
Chiarezza dei meccanismi di gioco (1–5)	4,75
Grafica (1–5)	4,75
Gradimento complessivo (1–5)	5,00
Divertimento (1–5)	5,00

Tabella 6: Risultati medi delle risposte raccolte con la versione 2.0.8.

Tutti i punteggi SUS ricadono nel grado più alto ("Eccellente") tra quelli mostrati dalla dashboard, e le valutazioni numeriche sono concentrate sui valori massimi della scala. Nei commenti liberi un partecipante indica come punto di forza un sistema di gioco "facile e comprensibile", mentre le criticità segnalate convergono su un unico aspetto: la gestione delle sfide, e in particolare della Battaglia, descritta come macchinosa e da ripensare.

Sulla base di questo riscontro la Battaglia, in precedenza sempre attiva, è diventata un'opzione della configurazione iniziale a partire dalla versione 2.0.12 (tabella 1, passo 5).

Capitolo 6

Conclusioni

6.1 Riepilogo del lavoro

Questo elaborato ha descritto la progettazione e lo sviluppo di una versione digitale del gioco da tavolo *La Città degli Imprevisti*, ideato dai partecipanti insieme all'équipe di educatori e psicologi del CFPIL di Varese come strumento di potenziamento delle competenze sociali per adolescenti con disturbo dello spettro autistico.

L'implementazione ha preso come punto di partenza un gioco fisico esistente — strutturato attorno a un percorso con caselle speciali che attivano sfide di natura sociale ed emotiva — e lo ha tradotto in un'applicazione web accessibile da qualsiasi browser moderno. Il lavoro si è sviluppato in due fasi: una prima versione (v1) puramente client-side, distribuita su GitHub Pages, e una versione corrente (v2) che introduce la modalità multi-dispositivo tramite API Routes, long polling e un database Redis. L'applicazione è in uso e continua a evolvere: le prime analisi dei feedback hanno già portato a modificare la Battaglia, resa configurabile.

6.2 Contributi

Il contributo principale del lavoro è la realizzazione di uno strumento pronto all'uso per le sessioni del CFPIL, che preserva la struttura del gioco originale e ne estende le possibilità operative. In particolare:

- la **modalità schermo singolo** riproduce fedelmente la dinamica del gioco da tavolo fisico, mantenendo l'interazione faccia a faccia e la mediazione dell'educatore;
- la **modalità multi-dispositivo** estende il gioco a contesti in cui non è disponibile un unico schermo condiviso, introducendo la gestione server-side delle sessioni e la distribuzione delle sfide tra dispositivi;

- l'**architettura evolutiva** dimostra come un'applicazione inizialmente statica possa essere estesa con funzionalità server-side senza riscrivere la logica di gioco, grazie alla separazione netta tra domain model, state management e presentation layer;
- la **valorizzazione dei partecipanti**: trasformare il prototipo cartaceo ideato dal gruppo in un prodotto tecnologico reale, presentato agli altri studenti del CFPIL, ha dato riconoscimento al lavoro dei ragazzi e ha reso lo strumento un mezzo relazionale condiviso;
- un **modello di gruppo che si sostiene autonomamente**: lo strumento è inserito in un contesto in cui i partecipanti più esperti fanno da *mentor* ai nuovi ingressi (osservazione qualitativa dell'équipe, non misurata).

Dal punto di vista del contesto applicativo, l'elaborato contribuisce a esplorare un approccio alla digitalizzazione del social skills training ancora poco studiato in letteratura: non un serious game sviluppato ex novo, ma la versione digitale di un gioco da tavolo nato e già utilizzato all'interno di una comunità educativa specifica, ideato dai suoi stessi destinatari.

6.3 Limitazioni

Il lavoro presenta alcune limitazioni che è opportuno esplicitare.

I dati di feedback raccolti sono pochi e riferiti alla sola versione 2.0.8 (sezione 5.2): riguardano la sola modalità multi-dispositivo e non comprendono docenti, e con un campione ridotto il punteggio SUS medio è poco stabile. Il SUS misura l'usabilità percepita, non l'efficacia formativa: non è quindi possibile, allo stato attuale, trarre conclusioni sull'efficacia dell'applicazione rispetto agli obiettivi educativi.

Sul piano della validità esterna, il testing è avvenuto in un singolo istituto, con un numero limitato di sessioni e senza un gruppo di controllo né una misurazione delle competenze sociali prima e dopo l'intervento. I risultati non sono generalizzabili senza ulteriori studi.

Come discusso nella sezione 4.8, la modalità multi-dispositivo introduce una tensione tra accessibilità e qualità dell'interazione sociale che non è stata misurata empiricamente.

Infine, la fase dedicata alla prevenzione del rischio sociale (sezione 6.4) è solo progettata: gli scenari e la sosta di discussione non sono implementati nell'applicazione né sperimentati. Anche gli effetti di valorizzazione personale e di *mentoring* descritti nel capitolo introduttivo sono osservazioni qualitative dell'équipe e non sono stati misurati.

6.4 Sviluppi futuri

Le direzioni di sviluppo più immediate riguardano la chiusura delle lacune identificate e l'ampliamento dei contenuti:

- **Ampliamento dei feedback** — raccogliere un numero maggiore di risposte, includendo docenti e sessioni a schermo singolo, e verificare con la dashboard (filtrando per versione) se la Battaglia configurabile ha risolto la criticità emersa nella prima raccolta;
- **Confronto tra modalità** — progettare una sessione sperimentale che confronti direttamente la modalità schermo singolo e la modalità multi-dispositivo in termini di qualità dell'interazione sociale osservata;
- **Estensione a più istituti** — coinvolgere altri centri che operano con utenza ASD per ampliare la base di test e valutare la portabilità dello strumento;
- **Accessibilità** — migliorare il supporto per screen reader e per dispositivi con capacità motorie ridotte, così da ampliare l'utenza potenziale;
- **Prevenzione del rischio sociale** — ampliare i contenuti del gioco con scenari di riconoscimento dei rischi sociali (bullismo, aggressioni verbali o fisiche, tentativi di raggiro e truffa) e di individuazione di strategie di tutela, mantenendo la struttura a caselle e scelte multiple. Un elemento centrale sarà la **sosta di discussione**: di fronte a una scelta complessa o a una divergenza di opinione, il gioco richiede ai giocatori di fermarsi a confrontarsi, stimolando il pensiero critico e costruendo un archivio di esperienze simulate a cui attingere nella vita reale. Il tema è rilevante: circa il 44% dei giovani con ASD subisce bullismo scolastico [11]. La fase, progettata con l'équipe e il gruppo, non è ancora implementata né sperimentata.
- **Validazione e alleggerimento delle animazioni facciali** — la transizione animata dal volto neutro all'espressione è già disponibile nel gioco, come opzione "Animato" disattivata di default, ma non è stata ancora sperimentata con l'utenza: non è noto se l'animazione favorisca il riconoscimento dell'emozione o, al contrario, distragga. Ogni gif pesa circa 7–8 MB (60 file, circa 450 MB complessivi), un costo non trascurabile su reti scolastiche limitate: andranno valutate una riduzione di risoluzione e di durata;
- **Riattivazione e ampliamento dei contenuti video** — la sfida "Film" è disattivata per l'assenza dei diritti sulle scene (sezione 4.5.1); per riattivarla occorre reperire scene con licenza libera o autorizzata, oppure produrle appositamente, ad esempio

con il gruppo stesso. In quell'occasione è opportuno andare oltre le cinque scene originali, una per emozione, che si ripetevano nelle partite lunghe, e dotarle di sottotitoli, migliorandone varietà e accessibilità.

Bibliografia

[1] Luca Surian. *L'autismo - Conoscerlo e affrontarlo*. Il Mulino, 2021. ISBN 978-88-15-29455-0.

[2] CFPIL sito ufficiale. https://agenziaformativa.va.it/sedi/varese-cfpil/, 2026.

[3] Gary B. Mesibov, Victoria Shea, and Eric Schopler. *The TEACCH Approach to Autism Spectrum Disorders*. Springer, 2005. ISBN 978-0-306-48647-0.

[4] Orla Walsh, Conor Linehan, and Christian Ryan. Is there evidence that playing games promotes social skills training for autistic children and youth? *Autism*, 29(2): 329–343, 2024. doi: 10.1177/13623613241277309. URL https://pmc.ncbi.nlm.nih.gov/articles/PMC11816463/.

[5] Gray Atherton and Liam Cross. The use of analog and digital games for autism interventions. *Frontiers in Psychology*, 12:669734, 2021. doi: 10.3389/fpsyg.2021.669734. URL https://pmc.ncbi.nlm.nih.gov/articles/PMC8384560/.

[6] Gabriele Mari. Gioco strutturato, gioco modificato: un'esperienza di giochi da tavolo con bambini e ragazzi con autismo. *L'integrazione scolastica e sociale*, 17(4):315–321, 2018. URL https://rivistedigitali.erickson.it/integrazione-scolastica-sociale/it/visualizza/pdf/1661.

[7] Ana Carneiro, Tânia Correia, Sara Felizardo, and Maria da Graça Ferreira. Serious games for developing social skills in children and adolescents with autism spectrum disorder: A systematic review. *Healthcare*, 12(5):508, 2024. doi: 10.3390/healthcare12050508.

[8] John Brooke. SUS: A quick and dirty usability scale. In Patrick W. Jordan, Bruce Thomas, Bernard A. Weerdmeester, and Ian L. McClelland, editors, *Usability Evaluation in Industry*. Taylor & Francis, 1996. ISBN 978-0-7484-0460-8.

[9] Natalie C. Ebner, Michaela Riediger, and Ulman Lindenberger. FACES—a database of facial expressions in young, middle-aged, and older women and men: Development and validation. *Behavior Research Methods*, 42(1):351–362, 2010. doi: 10.3758/BRM.42.1.351. URL `https://faces.mpdl.mpg.de/imeji/collection/IXTdg721TwZwyZ8e`.

[10] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. ISBN 978-0-201-63361-0.

[11] Christophe Maïano, Claude L. Normand, Marie-Claude Salvas, Grégory Moullec, and Annie Aimé. Prevalence of school bullying among youth with autism spectrum disorders: A systematic review and meta-analysis. *Autism Research*, 9(6):601–615, 2016. doi: 10.1002/aur.1568.